PLUGIN ARCHITECTURE



大型软件是怎么长出来的

用 C++ 从二十行代码到一个插件系统

从一段只会打印命令的程序开始

拆函数、拆文件、装插件、建契约，一步步长出可扩展的外壳

Dr. Dezeming

2026

PLUGIN ARCHITECTURE

大型软件是怎么长出来的

用 C++ 从二十行代码到一个插件系统

Dr. Dezeming

2026

大型软件是怎么长出来的

用 C++ 从二十行代码到一个插件系统

作者　Dr. Dezeming

版本　1.0　·　2026 年

本书从一段二十行的 C++ 程序开始，逐步拆函数、拆文件、加载动态库、建立契约、 处理事件与故障，最后做成一个可打包、可更新的插件系统。

书中代码示例采用 C++17 编写，只依赖标准库； 平台相关的部分会分别给出 Windows 与 Linux 的写法。

本书采用 CC BY-NC-SA 4.0 许可协议。

https://creativecommons.org/licenses/by-nc-sa/4.0/

目录

前言 1

写给谁 1

为什么用 C++ 1

路线 1

读法 1

第一篇 从一段程序开始 1

第 1 章 二十行程序 1

1.1 先让它跑起来 1

1.2 逐行读代码 1

1.3 编译到底做了什么 1

1.4 如果你的中文显示成乱码 1

1.5 它已经有一个内核的雏形 1

1.6 三个问题 1

1.7 下一步 1

第 2 章 把代码拆开 1

2.1 先把两段逻辑搬走 1

2.2 声明和定义是两件事 1

2.3 头文件：把“能做什么”集中起来 1

2.4 两个文件怎么变成一个程序 1

2.5 函数名就是符号 1

2.6 静态库和动态库 1

2.7 现在的结构还差什么 1

第 3 章 第一个插件 1

3.1 目标：不加新代码也能多一个命令 1

3.2 动态库到底是什么 1

3.3 先定一个最小的协议 1

3.4 为什么名字要按 C 的规则 1

3.5 函数指针：把函数当数据存起来 1

3.6 写一个插件 1

3.7 宿主：扫描、加载、调用 1

3.8 编译和运行 1

3.9 这一步解决了什么 1

3.10 还差什么 1

第二篇 把边界建起来 1

第 4 章 接口就是合同 2
4.1 第 3 章的插件还不会做事 2
4.2 先看一次完整的交接 2
4.3 为什么不能直接传 C++ 对象 2
4.4 用三个字段描述一个命令 2
4.5 把命令交出去的那个函数 2
4.6 工具箱：PluginApi 2
4.7 为什么结构体开头要放一个尺码 2
4.8 version 管的是意思有没有变 2
4.9 只给能力，不给内部数据 2
4.10 完整的接口头文件 2
4.11 插件这边怎么写 2
4.12 宿主这边怎么写 2
4.13 这一章定下了什么 2
第 5 章 注册表与扩展点 2
5.1 命令注册进去之后 2
5.2 一张能查、能删的表 2
5.3 执行一条命令 2
5.4 重名怎么办 2
5.5 卸载时清理 2
5.6 命令表不够用了 2
5.7 扩展点的三个要素 2
5.8 三种存法，各自的问题 2
5.9 用模板消除重复 2
5.10 三种扩展点形态 2
5.11 把这一章串起来 2
5.12 还差什么 2
第 6 章 事件与钩子 2
6.1 一次保存涉及哪些角色 2
6.2 事件长什么样 2
6.3 订阅：把函数地址交给总线 2
6.4 发布：总线挨个调用 2
6.5 顺序：谁先谁后 2
6.6 钩子：能改数据，也能阻止 2
6.7 钩子链：顺序、短路、数据 2
6.8 三种钩子 2
6.9 异常不能穿过边界 2
6.10 事件和钩子怎么配合 2
6.11 一个容易忽略的问题：重入 2
6.12 还差什么 2
第 7 章 插件出错怎么办 2
7.1 C++ 的崩溃为什么特别致命 2

7.2 程序运行起来之后 3
7.3 同一个进程里的保护为什么不够 3
7.4 把插件做成一个独立程序 3
7.5 两个程序怎么说话 3
7.6 超时与强制结束 3
7.7 权限：只给该给的 3
7.8 一个最小的进程插件 3
7.9 Linux 上对应的写法 3
7.10 什么时候不该隔离 3
7.11 这一章解决了什么 3

第三篇 把代码变成工程 3

第 8 章 一条命令生成插件 3
8.1 每写一个插件都要重复一遍 3
8.2 模板里放什么 3
8.3 目录结构 3
8.4 CMake 是什么 3
8.5 用脚本生成 3
8.6 模板长什么样 3
8.7 生成之后的检查 3
8.8 模板也会过时 3
8.9 模板不该做的事 3

第 9 章 让插件独立开发 3
9.1 独立开发是什么意思 3
9.2 把接口头文件单独放 3
9.3 假宿主 3
9.4 自己写一个测试框架 3
9.5 本地联调 3
9.6 持续集成 3
9.7 版本与依赖 3
9.8 插件作者的日常 3

第 10 章 打包、分发与更新 3
10.1 一个插件包长什么样 3
10.2 哈希：给文件算一个指纹 3
10.3 签名：确认是谁做的 3
10.4 安装、启用、禁用、卸载 3
10.5 更新的两种做法 3
10.6 热更新的边界 3
10.7 权限与安全 3
10.8 分发的成本 3

第 11 章 让它稳定运行 3
11.1 四类故障 3
11.2 超时 3

11.3 熔断与降级 4
11.4 C++ 的资源管理 4
11.5 内存泄漏 4
11.6 日志与诊断 4
11.7 版本演进 4
11.8 兼容性测试 4
11.9 故障之后 4
第四篇 收拢 **4**
第 12 章 把整个系统跑起来 **4**
12.1 目标 4
12.2 最终目录 4
12.3 每个文件负责什么 4
12.4 顶层构建文件 4
12.5 构建与运行 4
12.6 加一个新插件 4
12.7 故意制造错误 4
12.8 继续演进 4
12.9 最后 4
附录 A 术语表 **4**
程序与编译 4
插件与接口 4
运行与隔离 4
工程与发布 4
附录 B 延伸阅读 **4**

前言

曾经的我，以能维护几十万行代码的引擎为荣，各个模块都组织得井井有条。而如今，AI 却能一口气写出一个引擎并完成测试，这让我感到前所未有的震撼与迷茫。

在全民 Vibe Coding 的时代，我们还需要掌握软件开发技术吗？答案是肯定的。这一年里，我接手的项目大多是 Vibe Coding 产出的屎山代码；其中，Codex 生成的代码尤为典型，很多问题都源于 Skill 和提示词使用不规范。 这些屎山代码或多或少都已处在崩溃边缘，不得不让我重新思考软件工程的意义：当前的 AI 真的能完全取代软件工程能力吗？实则不然。具备一定的软件技术基础，才能让我们构建出更健壮、更强大的大型系统。

先看一段程序。

```cpp
#include <iostream>
#include <string>

int main() {
    std::string line;
    while (true) {
        std::cout << "> ";
        if (!std::getline(std::cin, line)) break;
        if (line == "quit") break;
        if (line == "hello") {
            std::cout << "你好，世界\n";
        } else if (line == "time") {
            std::cout << "现在该干活了\n";
        } else {
            std::cout << "不认识的命令: " << line << "\n";
        }
    }
    return 0;
}
```

它只有二十来行，运行起来是一个不断等待输入的小窗口。你输入 `hello`，它回一句问候；输入 `time`，它回一句提醒；输入 `quit`，它退出。

这本书要从这段程序出发，一步一步把它变成一个大型软件的样子：功能由不同的文件提供，新功能可以单独编译、单独发布，第三方写的模块能在不重新编译主程序的情况下装进来，某个模块出错时主程序还能活着。

这个过程会经过很多次改造。每改造一次，只增加一个概念，并且给出能编译、能运行的代码。你不会看到“定义一个接口然后动态加载实现”这种一句话带过的说法，因为那句话背后至少有六件事要处理，每一件都值得单独讲一章。

写给谁

这本书假定你学过一門编程语言的基础语法，知道变量、函数、循环、数组大概是什么，用 C++ 写过程序，哪怕只写过一个控制台小程序。你不需要懂 Java，不需要学过操作系统，也不需要知道编译原理。

书里会反复出现一些工程词，比如编译、链接、动态库、进程、序列化、事件、钩子。这些词第一次出现时都会先用日常经验解释，再放进代码里。如果某个词你在别处看过但一直没弄懂，这本书希望把它讲到你能够自己复述出来。

为什么用 C++

C++ 有一个好处，也有一个坏处。

好处是它把很多被高级语言藏起来的东西摆在明面上。你可以看到头文件和源文件是分开的，可以看到编译和链接是两个步骤，可以看到动态库在运行时才被加载。这些机制在 Java、C#、Python 里也有，但被包装得更深，初学时不容易看清。

坏处是 C++ 的细节多，容易写出崩溃的程序。书里会尽量把代码控制在标准库范围内，不引入第三方框架，每一段都尽量短。涉及平台差异的地方，会分别给出 Windows 和 Linux 的写法。

路线

全书分四篇，十二章。

第一篇从那段二十行的程序开始。先把它拆成函数，再拆成多个文件，然后编译成动态库，最后让主程序在运行时把动态库读进来。这一篇结束时，你已经有第一个真正意义上的插件。

第二篇处理边界。插件和主程序之间要有合同，合同要能版本化；主程序不能认识每一个插件的名字，要用注册表和扩展点；插件之间要通过事件和钩子协作；插件崩溃时，主程序要能活下来。

第三篇把它变成工程。用一条命令生成新组件的骨架，让组件可以脱离主程序单独开发和测试，再打包、签名、分发、更新。

第四篇收拢。把前面所有代码拼成一个可以照着搭起来的最小系统，给出完整目录、构建命令和一份继续演进清单。

读法

书里的代码都可以直接编译。建议你至少动手跑一遍前三章的代码，因为“动态库被加载进来”这件事，看文字和亲眼看它跑起来，理解程度差别很大。

如果某一段代码暂时看不懂，可以先跳过细节，看它前后的中文说明，知道这段在整个流程里负责什么，再回头逐行读。工程的难点往往不在某一行语法，而在于很多行怎么配合。

准备好了的话，从第一段程序开始。

第一篇 从一段程序开始

这一篇不谈架构，只做一件事：把一段能跑的小程序，一步步改造成能加载外部模块的程序。每一步都对应一个真实的工程问题，也对应一个 C++ 里本来就存在的机制。

第 1 章逐行读完那段二十行的程序，指出它已经具备的三个零件。

第 2 章把代码拆成函数、头文件和多个源文件，讲清楚编译和链接。

第 3 章把一部分代码编译成动态库，让主程序在运行时加载它，得到第一个插件。

第 1 章 二十行程序

本章的完整代码和编译说明在 `codes/ch01/`。

1.1 先让它跑起来

把下面这段代码保存成一个文件，名字叫 `main.cpp`。

```
#include <iostream>
#include <string>

#ifdef _WIN32
#include <windows.h>
#endif

int main() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    std::string line;
    while (true) {
        std::cout << "> ";
        if (!std::getline(std::cin, line)) break;
        if (line == "quit") break;
        if (line == "hello") {
            std::cout << "你好，世界\n";
        } else if (line == "time") {
            std::cout << "现在该干活了\n";
        } else {
            std::cout << "不认识的命令：" << line << "\n";
        }
    }
    return 0;
}
```

在 Windows 上，如果你装的是 Visual Studio，打开“开发者命令提示”窗口，进入这个文件所在的目录，输入：

```
cl /EHsc /std:c++17 main.cpp
```

会得到一个 `main.exe`，双击或者在命令行里输入 `main.exe` 就能运行。

如果你用的是 MinGW 或者 Linux，输入：

```
g++ -std=c++17 main.cpp -o app
./app
```

运行之后，屏幕上会出现一个 `>`，等你输入。输入 `hello` 回车，它回答“你好，世界”；输入 `time` 回车，它说“现在该干活了”；输入别的，它说不认识；输入 `quit`，程序结束。

这一章剩下的部分，就是把这段代码一行一行拆开，看清楚它到底在做什么。别急着往后翻，先确认你能把它跑起来。

1.2 逐行读代码

开头的几个 include：把工具箱拿进来

```
#include <iostream>
#include <string>

#ifdef _WIN32
#include <windows.h>
#endif
```

`#include` 的意思是“把另一个文件的内容搬到这里”。尖括号里的 `iostream` 和 `string` 是 C++ 标准库自带的两个工具箱。

`iostream` 里装着输入输出的工具，屏幕打印、键盘读取都要用它。`string` 里装着字符串工具，也就是“一串文字”。

`#ifdef _WIN32` 到 `#endif` 之间是条件编译：只有在 Windows 上编译时，才会把 `windows.h` 拿进来。这个头文件提供 `SetConsoleOutputCP`，用来设置控制台编码，1.4 节会解释。在 Linux 或 macOS 上，这三行会被跳过，不参与编译。

main：程序的入口

```
int main() {
```

`main` 是一个特殊函数。操作系统启动一个程序时，会先找到这个名字，从它开始执行。`int` 表示这个函数最后要返回一个整数，`()` 表示它不接收参数。

你可以把 `main` 想成一家店的正门。客人从正门进来，店里的事从这里开始；正门关着，店就没法营业。

准备一个装文字的盒子

```
    std::string line;
```

这一行声明了一个变量，名字叫 `line`，类型是 `std::string`，也就是“一串文字”。它现在是一个空盒子，等着装用户输入的内容。

`std::` 是一个前缀，表示“这个东西来自标准库”。C++ 标准库把所有东西都放在 `std` 这个命名空间里，避免和你的变量重名。写起来麻烦一点，但不容易撞车。

不停地问

```
    while (true) {
```

`while` 是循环。括号里的条件为真，就重复执行大括号里的内容。`true` 永远为真，所以这是一个“一直转下去”的循环，直到遇到 `break` 才停。

大型软件的主界面也是这样的循环：等用户操作，处理，再等下一个操作。这个循环有个名字，叫**事件循环**。现在你看到的版本很简单，只有键盘输入；真实的软件还要处理鼠标、窗口大小变化、网络消息，但骨架是同一个。

```
        std::cout << "> ";
```

`std::cout` 是标准输出，默认就是屏幕。`<<` 把右边的东西送到左边。这一行在屏幕上打印一个 `>` 和一个空格，作为提示符。

```
        if (!std::getline(std::cin, line)) break;
```

`std::cin` 是标准输入，默认就是键盘。`std::getline(std::cin, line)` 从键盘读一整行，放进 `line` 这个盒子里。

它还有一个返回值，表示“读到了没有”。读到返回真，读不到返回假。前面的 `!` 是“取反”，所以整句的意思是：如果没读到，就跳出循环。

什么时候会读不到？用户按下 Ctrl+Z（Windows）或者 Ctrl+D（Linux），表示输入结束；或者输入流出了别的问题。没有这一行，程序在输入结束时可能陷入死循环。

```
        if (line == "quit") break;
```

如果用户输入的是 `quit`，跳出循环，程序往下走到结尾。

`==` 是比较，判断两边是否相等。注意是两个等号，一个等号是赋值，这是初学者最容易写错的地方。

```
        if (line == "hello") {
            std::cout << "你好，世界\n";
        } else if (line == "time") {
            std::cout << "现在该干活了\n";
        } else {
            std::cout << "不认识的命令：" << line << "\n";
        }
```

这是一串条件判断。先看是不是 `hello`，是就打印问候；不是再看是不是 `time`，是就打印提醒；都不是，就打印“不认识”，并把用户输入的内容原样带出来。

`\n` 是换行符。没有它，下一次输出会接在同一行后面。

关门

```
    return 0;
}
```

`return 0` 表示程序正常结束。操作系统会收到这个 0，认为“一切正常”。如果程序因为错误退出，通常返回一个非零的数字，方便脚本或者启动器判断。

1.3 编译到底做了什么

你输入的那条 `cl` 或者 `g++` 命令，背后做了两件事。

第一件叫**编译**。编译器读 `main.cpp`，检查语法对不对，把每一行翻译成计算机能执行的指令，生成一个中间文件。Visual Studio 生成的是 `main.obj`，MinGW 和 Linux 生成的是 `main.o`。

第二件叫**链接**。你的程序用到了 `std::cout`、`std::string` 这些东西，它们的实现不在你的文件里，而在标准库里。链接器负责把你的 `main.obj` 和标准库拼在一起，生成最终的 `main.exe` 或者 `app`。

可以借做饭来理解。编译是把菜谱翻译成一份厨房能执行的工序单，链接是把工序单和厨房里现成的调料、厨具配齐。工序单写得再对，缺了调料也做不出菜。

现在你不需要记住所有细节，只要知道两件事。第一，源文件不是直接运行的，中间要经过翻译。第二，你的程序可以引用别处写好的东西，链接器会把它们接上。

第 2 章会把这两个步骤拆开，让你亲眼看到中间文件长什么样。

1.4 如果你的中文显示成乱码

Windows 的控制台默认用 GBK 编码显示文字，而 C++ 源文件现在普遍是 UTF-8。编译器把源码里的中文按 UTF-8 编进程序，控制台却按 GBK 去解释，屏幕上就会出现“浣犲ソ”这样的乱码。

代码里那三行 `SetConsoleOutputCP(CP_UTF8)` 就是解决这个问题的。它在程序启动时把控制台的输出编码切到 UTF-8，和源码保持一致。

如果你在 Linux 或者 macOS 上编译，这几行会被 `#ifdef _WIN32` 挡掉，不参与编译。那些系统的终端默认就是 UTF-8，不需要额外设置。

本书后面所有示例都做了同样的设置，代码里不再重复解释。

1.5 它已经有一个内核的雏形

这段程序虽然小，但已经包含了三个零件，后面所有章节都在扩展它们。

一个循环。 `while (true)` 不停地等待输入、处理、输出。这是所有交互式程序的心脏。

一张命令表。 现在这张表是 `if` 和 `else if` 堆出来的：`hello` 对应一段代码，`time` 对应另一段。表在代码里，写死了。

一个输出通道。 `std::cout` 把结果送到屏幕。现在只有一个通道，将来可能有窗口、日志文件、网络。

把这三个零件抽出来看，一个插件系统的骨架就浮出来了。循环是宿主的主循环，命令表是注册表，输出通道是宿主提供的能力。后面要做的，是把写死的命令表变成可以运行时添加的，把一段段代码从 `main` 里搬出去，让它们可以单独编译、单独替换。

1.6 三个问题

这段程序能跑，但每加一个功能，你都会难受一次。

加一个命令，要改主程序。 想加一个 `date` 命令，你得打开 `main.cpp`，在 `if` 链里再插一段，重新编译整个程序。如果这个程序是别人装的，你还得让所有人重新下载。命令越多，`main` 越长，最后没人敢动。

所有代码在同一个文件里。 现在只有二十行，无所谓。等它变成两千行，函数、变量、输出混在一起，找一个地方要翻半天。更麻烦的是，你没法把某一部分单独拿出去测试，因为它依赖 `main` 里的局部变量。

一个错误会带走整个程序。 C++ 没有 Java 那样的自动内存检查。一个数组越界、一个空指针、一次除零，都可能让程序直接崩溃，用户没保存的东西一起消失。如果所有功能都在一个程序里，任何一个功能的错误都是整个软件的错误。

这三个问题对应三个改造方向：**把代码搬出去、把加载变成运行时、把错误关进笼子**。

第一篇先解决第一个和第二个。第三个留到第 7 章。

1.7 下一步

改造从最小的一步开始：把 `hello` 和 `time` 这两段处理逻辑，从 `main` 里搬出来，变成两个函数。

这一步看起来只是“让代码整齐一点”，但它会带来一个关键的变化：函数有了名字，就可以被别的地方调用；可以被别的地方调用，就有可能被别的文件调用；能被别的文件调用，就有可能被编译成独立的模块。

从函数到文件，从文件到动态库，从动态库到插件，这条路是一步步走出来的。下一章走第一步。

第 2 章 把代码拆开

本章的完整代码和编译说明在 `codes/ch02/`。

2.1 先把两段逻辑搬走

第 1 章的程序里，`hello` 和 `time` 的处理逻辑直接写在 `if` 链里。现在把它们搬出来，各变成一个函数。

```
#include <iostream>
#include <string>

void cmd_hello() {
    std::cout << "你好，世界\n";
}

void cmd_time() {
    std::cout << "现在该干活了\n";
}

int main() {
    std::string line;
    while (true) {
        std::cout << "> ";
        if (!std::getline(std::cin, line)) break;
        if (line == "quit") break;
        if (line == "hello") {
            cmd_hello();
        } else if (line == "time") {
            cmd_time();
        } else {
            std::cout << "不认识的命令：" << line << "\n";
        }
    }
    return 0;
}
```

`void cmd_hello()` 里的 `void` 表示这个函数不返回任何值，`()` 表示它不接收参数。函数体就是大括号里的内容。

`main` 里原来的两段输出，变成了两次函数调用。程序的行为完全一样，但结构变了：处理逻辑有了名字，`main` 只负责“判断用户输入的是哪个命令，然后调用对应的函数”。

这一步看起来只是整理，但它带来了一个关键变化。函数有了名字，就可以被别的地方调用；能被别的地方调用，才有可能被搬到别的文件里。

2.2 声明和定义是两件事

把函数搬到另一个文件之前，要先分清两个概念。

定义是函数的完整实现，包括函数体：

```
void cmd_hello() {
    std::cout << "你好，世界\n";
}
```

声明只告诉编译器“有这么个函数，它长这样”，不写函数体：

```
void cmd_hello();
```

声明以分号结尾，没有大括号。它相当于一张名片：告诉你这个人叫什么、能接受什么、会返回什么，但不会告诉你他具体怎么干活。

为什么需要名片？因为 C++ 编译器是“从上往下、一个文件一个文件”处理的。当它在 `main` 里看到 `cmd_hello()` 时，必须已经知道这个名字是个函数、不接受参数、不返回值。如果函数定义写在后面，编译器读到调用时还没见过它，就会报“未声明的标识符”。

有两种解决办法。一种是把定义挪到调用前面，但函数一多，顺序就很难安排，A 调用 B、B 调用 A 时没法同时满足。另一种是把声明集中放在一个文件里，谁需要就把它拿过去。这个文件就是头文件。

2.3 头文件：把“能做什么”集中起来

新建一个文件 `commands.h`：

```
#pragma once

void cmd_hello();
void cmd_time();
```

`#pragma once` 是一句给编译器的叮嘱：这个文件在一次编译里只读一遍。为什么需要它？因为一个头文件可能被多个文件引用，也可能被间接引用多次。没有这句话，同一个声明可能出现好几遍，编译器会抱怨“重复定义”。

严格说，重复声明同一个函数在 C++ 里是允许的，但头文件里往往还有类型、常量、宏，这些东西重复了就会出错。加上 `#pragma once` 是最省事的做法，主流编译器都支持。

现在建第二个文件 `commands.cpp`，把两个函数的定义放进去：

```
#include <iostream>
#include "commands.h"

void cmd_hello() {
    std::cout << "你好，世界\n";
}

void cmd_time() {
    std::cout << "现在该干活了\n";
}
```

注意 `#include "commands.h"` 用的是双引号。双引号表示“先在我自己所在的目录找”，尖括号表示“去系统或者编译器配置的目录找”。自己写的头文件用双引号，标准库用尖括号。

`main.cpp` 也改成引用这个头文件：

```
#include <iostream>
#include <string>
#include "commands.h"

int main() {
    std::string line;
    while (true) {
        std::cout << "> ";
        if (!std::getline(std::cin, line)) break;
        if (line == "quit") break;
        if (line == "hello") {
            cmd_hello();
        } else if (line == "time") {
            cmd_time();
        } else {
```

```
            std::cout << "不认识的命令: " << line << "\n";
        }
    }
    return 0;
}
```

现在 `main.cpp` 里没有 `cmd_hello` 的实现，只有它的名片。实现藏在 `commands.cpp` 里。这就是最初级的“接口与实现分离”：`main.cpp` 知道能做什么，不知道怎么做。

2.4 两个文件怎么变成一个程序

编译命令要同时列出两个源文件。

Visual Studio 的开发者命令行：

```
cl /EHsc /std:c++17 main.cpp commands.cpp
```

MinGW 或者 Linux：

```
g++ -std=c++17 main.cpp commands.cpp -o app
```

编译器其实做了两步。

第一步，把每个 `.cpp` 单独编译成一个中间文件。`main.cpp` 编译成 `main.obj`，`commands.cpp` 编译成 `commands.obj`。这一步只检查语法，不关心 `cmd_hello` 到底在哪里实现。编译器看到声明，就假设“别处一定有”，先放行。

第二步，链接器把两个 `.obj` 和标准库拼在一起。它发现 `main.obj` 里有一个对 `cmd_hello` 的调用，就去别的 `.obj` 里找这个名字。在 `commands.obj` 里找到了，就把调用地址填上。如果找不到，就报“无法解析的外部符号”。

这里有一个很重要的概念，叫**编译单元**。一个 `.cpp` 文件加上它 `#include` 进来的所有头文件，构成一个编译单元。编译器一次处理一个编译单元，处理完就忘掉，不会记住别的 `.cpp` 里有什么。

所以头文件的作用不是“共享代码”，而是“共享声明”。真正的代码仍然在各自的 `.cpp` 里，链接器负责把它们接起来。

可以用盖房子来理解。每个 `.cpp` 是一个施工队，各自按图纸施工，不需要知道别的队在干什么。头文件是图纸上的接口说明：这里要留一个门，尺寸多少。链接器是最后验收的人，检查每个门是不是都有人装上了。

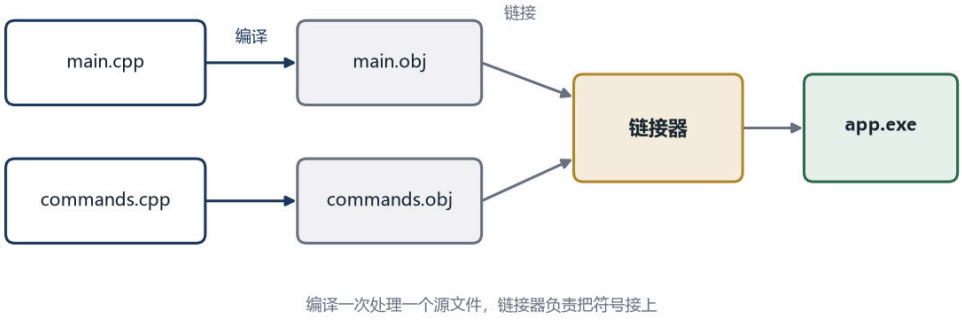


图 2-1　两个源文件经过编译和链接变成一个可执行文件

2.5 函数名就是符号

链接器靠什么找函数？靠**符号**。编译器会把函数名变成一个符号，写进 `.obj` 文件里。

在 C 语言里，函数 `cmd_hello` 的符号就是 `cmd_hello`，很直白。C++ 支持函数重载，也就是同名函数可以有不同参数，所以编译器要把参数信息也编进符号里。`void cmd_hello()` 可能变成 `?cmd_hello@@YAXXZ` 这样的名字。这个过程叫**名字修饰**。

名字修饰保证了同名不同参数的函数能被区分开，但也带来一个问题：不同编译器修饰出来的名字可能不一样。C++ 标准没有规定具体的修饰规则，Visual Studio 和 GCC 就各用各的。这意味着用 Visual Studio 编译的 `.obj`，不一定能和 GCC 编译的 `.obj` 链接在一起。

普通项目里这不是问题，因为所有人用同一套工具链。但插件系统要面对“插件可能是别人用别的编译器做的”这种情况，所以需要一种稳定的、跨编译器的符

号名。答案是用 `extern "C"`，下一章会用到。

2.6 静态库和动态库

现在有两个 `.obj`，可以每次编译都把它们列出来。文件多了之后，这样很麻烦。于是有了**库**。

静态库是把一堆 `.obj` 打包成一个文件。Windows 上是 `.lib`，Linux 上是 `.a`。链接的时候，链接器从这个包里挑出需要的 `.obj`，复制进最终的程序。

静态库的好处是简单，程序拿到就能跑，不依赖外部文件。坏处是每个用到它的程序都带着一份副本，体积变大；库更新了，程序必须重新链接才能用上新版本。

动态库是另一种打包方式。Windows 上是 `.dll`，Linux 上是 `.so`。它不复制进程序，而是在程序运行时才被加载。多个程序可以共用同一份动态库；库更新了，只要接口没变，程序不用重新编译。

动态库正是插件系统的基础。主程序可以在启动时扫描一个目录，把里面的动态库一个个加载进来，调用它们提供的函数。加一个新功能，只需要放一个新的动态库进去，主程序一行都不用改。

第 3 章就做这件事。



图 2-2　静态库在链接时复制，动态库在运行时加载

2.7 现在的结构还差什么

把代码拆成函数和文件之后，程序好读了一些，但三个问题只解决了一个半。

“加功能要改主程序”这个问题还在。加一个 `date` 命令，仍然要在 `commands.cpp` 里加一个函数，在 `commands.h` 里加一行声明，在 `main.cpp` 的 `if` 链里加一个分支，然后重新编译整个程序。

“所有代码在一个文件”解决了一半。逻辑分开了，但最终还是链接成同一个可执行文件，运行时仍然是一整块。

“一个错误带走整个程序”完全没解决。

要真正解决第一个问题，需要让主程序在运行时才发现新功能。这要求功能不在编译时链接进来，而是在运行时被加载。这就是动态库要做的事。

第 3 章 第一个插件

本章的完整代码和编译说明在 `codes/ch03/`。

3.1 目标：不加新代码也能多一个命令

现在的程序，每加一个命令都要改三个地方：`commands.h` 加声明，`commands.cpp` 加实现，`main.cpp` 的 `if` 链加分支，然后重新编译。

这次换一种做法。主程序只负责“找到插件、加载插件、调用插件”，具体有什么命令，由插件自己决定。加一个新命令，只需要往 `plugins` 目录里放一个新的动态库，主程序不动。

这一章会写出两个东西。一个是很小的插件，提供一个 `hello` 命令；一个是宿主，负责把插件目录里的动态库读进来。代码不长，但涉及几个新概念，每个都会解释。

3.2 动态库到底是什么

第 2 章说过，动态库在程序运行时才被加载。说得再具体一点。

一个 `.dll` 或者 `.so` 文件，本质上是一个装好了代码的盒子。盒子里有编译好的机器指令，还有一张**导出表**，写着“外面可以调用我这里的哪些函数，它们叫什么名字、在什么位置”。

宿主程序拿到这个盒子，先问操作系统要一个“把手”，然后通过把手去查导出表，找到某个函数的地址，把它存进一个函数指针，之后就能像调用普通函数一样调用它。

这个过程和去仓库取货很像。宿主不需要知道货放在哪个架子，只需要把货号报给仓库管理员，管理员把货递出来。动态库是仓库，操作系统是管理员，导出表

是货号清单。

3.3 先定一个最小的协议

插件和宿主之间要有约定，否则宿主不知道插件里有什么函数、该怎么调用。这一章先用一个最小的约定，三个函数：

```
// plugin_api.h
#pragma once

#ifdef _WIN32
#define PLUGIN_EXPORT __declspec(dllexport)
#else
#define PLUGIN_EXPORT __attribute__((visibility("default")))
#endif

extern "C" {
    PLUGIN_EXPORT const char* plugin_name();
    PLUGIN_EXPORT int plugin_init();
    PLUGIN_EXPORT void plugin_shutdown();
}
```

`plugin_name` 返回插件的名字，宿主用它来显示。`plugin_init` 在插件被加载后调用，插件可以在这里做初始化。`plugin_shutdown` 在卸载前调用，插件在这里清理自己占用的东西。

`PLUGIN_EXPORT` 是一个宏。在 Windows 上，它展开成 `__declspec(dllexport)`，意思是“这个函数要写进导出表，外面可以调用”。在 Linux 上，它展开成 `__attribute__((visibility("default")))`，意思类似：这个函数对外可见。

宏可以理解成“编译前的文本替换”。`#define` 定义一个名字和一段文本，编译器读到这个名字就换成那段文本。这里用它来抹平两个平台的写法差异，避免在代码里到处写 `#ifdef`。

3.4 为什么名字要按 C 的规则

第 2 章说过，C++ 会对函数名做修饰，把参数信息编进符号里。不同编译器修饰规则不同，甚至同一编译器的不同版本也可能不同。

插件是别人编译的，可能用 Visual Studio，也可能用 GCC。如果宿主按修饰后的名字去找函数，换个编译器就找不到了。所以插件接口要用 `extern "C"`，告诉编译器“这几个函数按 C 的规则来，名字不要修饰”。

`extern "C"` 只影响链接时的名字，不影响函数内部的写法。函数体里仍然可以写 C++，可以用类、模板、标准库。它只保证这个名字在外面看起来是稳定的。

生活里可以这样理解。C++ 的名字修饰像给每个人起了一个包含生日、籍贯、血型的长名字，确保不会重名，但外人很难记住。`extern "C"` 是说“这个人对外就用身份证上的名字”，简单、稳定、跨系统都能查。

3.5 函数指针：把函数当数据存起来

宿主从动态库里取出来的东西，不是一个能直接调用的函数，而是一个地址。要调用它，得先把地址存进**函数指针**。

函数指针的写法有点绕：

```
const char* (*name_fn)();
```

从里往外读。`(*name_fn)` 表示 `name_fn` 是一个指针。右边的 `()` 表示它指向的函数不接受参数。左边的 `const char*` 表示那个函数返回一个字符串。合起来：`name_fn` 是一个指向“无参数、返回字符串”的函数的指针。

调用的时候写 `name_fn()`，和普通函数一样。编译器知道它是个指针，会跳到那个地址去执行。

这就像你把朋友的电话号码存进通讯录。号码本身不是朋友，但拨这个号码就能找到他。函数指针就是函数的号码。

3.6 写一个插件

新建 `hello_plugin.cpp`：

```cpp
#include "plugin_api.h"
#include <cstdio>

static int counter = 0;

extern "C" PLUGIN_EXPORT const char* plugin_name() {
    return "hello";
}

extern "C" PLUGIN_EXPORT int plugin_init() {
    std::printf("[hello] init\n");
    std::fflush(stdout);
    return 0;
}

extern "C" PLUGIN_EXPORT void plugin_shutdown() {
    std::printf("[hello] shutdown, counter=%d\n", counter);
    std::fflush(stdout);
}
```

`static int counter = 0;` 是插件自己的状态，记录这个插件被调用了多少次。`static` 放在全局变量前面，表示“这个变量只在本文件内可见”，别的文件看不到，也不会和别人的同名变量冲突。

`plugin_init` 返回 0 表示初始化成功，返回非零表示失败。宿主会根据这个返回值决定要不要继续使用这个插件。

`std::printf` 是 C 风格的输出，`std::fflush(stdout)` 把缓冲区里的内容立刻送到屏幕。为什么不用 `std::cout`？因为动态库和主程序可能各自持有一份输出缓冲，混用时顺序容易乱。`printf` 配合刷新更稳，代码也更短。

3.7 宿主：扫描、加载、调用

宿主需要做四件事：找到动态库、打开它、取出三个函数、按顺序调用。

```cpp
// main.cpp
#include <cstdio>
#include <filesystem>
#include <string>
#include <vector>

#ifdef _WIN32
#include <windows.h>
using LibHandle = HMODULE;
static LibHandle open_lib(const std::string& p) { return LoadLibraryA(p.c_str()); }
static void* get_sym(LibHandle h, const char* n) { return (void*)GetProcAddress(h,
```

```
n); }
static void close_lib(LibHandle h) { FreeLibrary(h); }
#else
#include <dlfcn.h>
using LibHandle = void*;
static LibHandle open_lib(const std::string& p) { return dlopen(p.c_str(),
RTLD_NOW); }
static void* get_sym(LibHandle h, const char* n) { return dlsym(h, n); }
static void close_lib(LibHandle h) { dlclose(h); }
#endif

int main() {
    std::vector<LibHandle> loaded;
    for (const auto& entry : std::filesystem::directory_iterator("plugins")) {
#ifdef _WIN32
        if (entry.path().extension() != ".dll") continue;
#else
        if (entry.path().extension() != ".so") continue;
#endif
        LibHandle h = open_lib(entry.path().string());
        if (!h) { std::printf("load failed: %s\n", entry.path().string().c_str());
continue; }

        auto name = (const char*(*)())get_sym(h, "plugin_name");
        auto init = (int(*)())get_sym(h, "plugin_init");
        if (!name || !init) { std::printf("missing symbols\n"); close_lib(h);
continue; }

        std::printf("loaded %s\n", name());
        std::fflush(stdout);
        init();
        loaded.push_back(h);
    }

    for (auto h : loaded) {
        auto shut = (void(*)())get_sym(h, "plugin_shutdown");
        if (shut) shut();
        close_lib(h);
    }
    return 0;
}
```

一段一段看。

`using LibHandle = HMODULE;` 给类型起别名。Windows 上动态库的把手是 `HMODULE`，Linux 上是 `void*`。用同一个名字 `LibHandle`，后面的代码就不用到处写平台判断。

`open_lib` 打开动态库。Windows 用 `LoadLibraryA`，参数是文件路径；Linux 用 `dlopen`，第二个参数 `RTLD_NOW` 表示“现在就把需要的符号都解析好”。失败时返回空。

`get_sym` 按名字取函数地址。Windows 用 `GetProcAddress`，Linux 用 `dlsym`。返回值是 `void*`，需要强制转换成具体的函数指针类型。

`close_lib` 关闭动态库，释放它占用的资源。

`std::filesystem::directory_iterator("plugins")` 遍历 `plugins` 目录下的每一个条目。这是 C++17 标准库提供的文件系统功能，不需要额外装库。

`entry.path().extension()` 取文件扩展名。Windows 只处理 `.dll`，Linux 只处理 `.so`。

`auto name = (const char*(*)())get_sym(...)` 把取到的地址转换成函数指针。`auto` 表示“类型由右边推断”，省得手写一长串。

`if (!name || !init)` 检查必需的两个函数在不在。缺了就跳过这个库，并打印提示。注意这里没有检查 `plugin_shutdown`，因为它是可选的，卸载时再取一次就行。

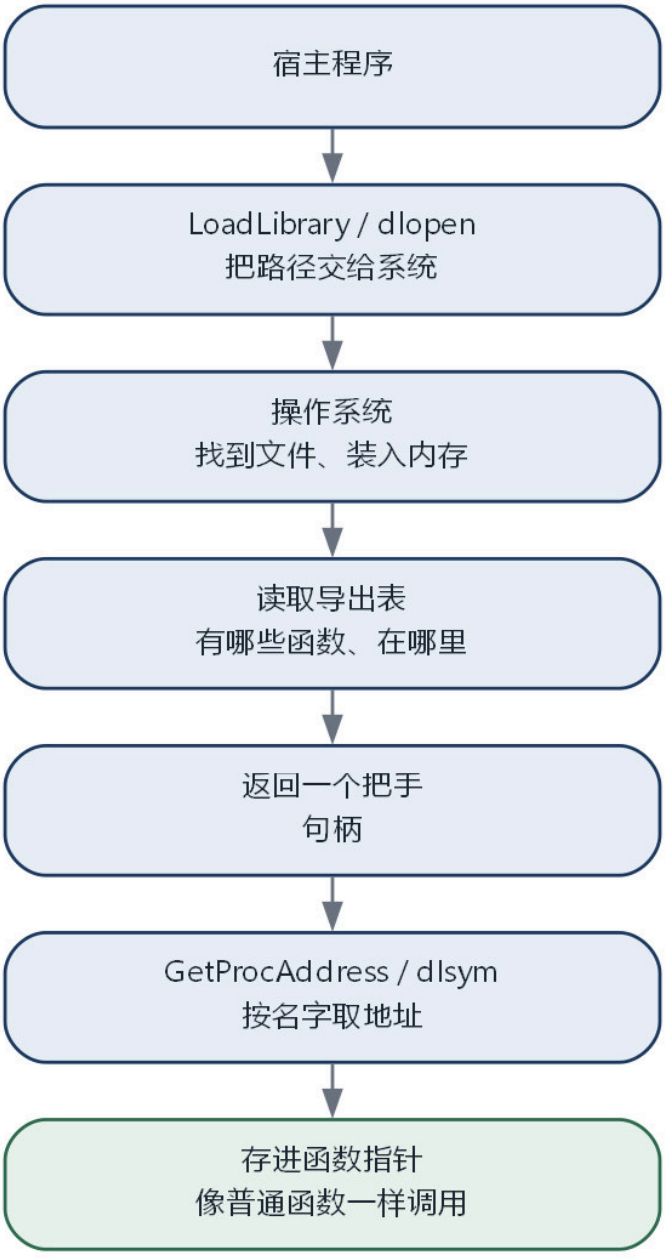


图 3-1　动态库从文件到函数调用经过的步骤

3.8 编译和运行

Windows 的开发者命令行：

```
cl /EHsc /std:c++17 /LD hello_plugin.cpp /Fe:plugins\hello_plugin.dll
cl /EHsc /std:c++17 main.cpp /Fe:host.exe
host.exe
```

第一行编译插件。`/LD` 表示“生成动态库”，`/Fe:` 指定输出路径。运行后 `plugins` 目录里会多出 `hello_plugin.dll`，可能还会多一个 `.lib` 和一个 `.exp`，那是链接时的附带产物，可以忽略。

第二行编译宿主。第三行运行。

Linux 或者 MinGW：

```
mkdir -p plugins
g++ -std=c++17 -shared -fPIC hello_plugin.cpp -o plugins/hello_plugin.so
g++ -std=c++17 main.cpp -o host -ldl
./host
```

`-shared` 表示生成动态库，`-fPIC` 表示生成位置无关的代码，动态库需要这个选项。`-ldl` 表示链接动态加载相关的库。

运行后应该看到：

```
loaded hello
[hello] init
[hello] shutdown, counter=0
```

三行分别来自宿主、插件的初始化、插件的清理。顺序说明宿主先拿到名字，再调用初始化，最后在退出前调用清理。

现在往 `plugins` 目录里再放一个动态库，宿主会把它也加载进来，主程序一行都不用改。这就是插件系统最核心的能力。

3.9 这一步解决了什么

回头看第 1 章提出的三个问题。

“加功能要改主程序”解决了一大半。新功能现在是一个独立的动态库，主程序只负责加载。虽然主程序里的 `if` 链还在，但它已经不需要认识具体命令了，因为它根本不知道插件里有什么命令。

“所有代码在一个文件”解决了。插件可以单独编译、单独发布，甚至可以由别人来写。

“一个错误带走整个程序”还没解决。插件和宿主在同一个进程里，插件里一个空指针照样能让整个程序崩溃。这个问题要到第 7 章。

3.10 还差什么

现在的插件协议很脆弱，有几个明显的问题。

只有三个函数，插件没法注册命令。 宿主加载了插件，但不知道插件能做什么。需要一个机制，让插件把自己的命令交给宿主。

没有版本。 宿主怎么知道这个插件是给哪个版本的接口写的？接口一改，旧插件可能直接崩溃。

没有隔离。 插件可以访问宿主的任何内存，也可以调用系统的任何功能。一个恶意插件可以为所欲为。

没有清理。 插件注册的东西，卸载时怎么撤销？现在的 `plugin_shutdown` 只是一个通知，宿主没有统一的登记表。

加载顺序不确定。 目录遍历的顺序不固定，如果插件之间有依赖，谁先谁后就成了问题。

接下来的三章，先把前两个问题解决：用注册表让插件交出命令，用契约把接口固定下来。隔离和清理放到第二篇和第三篇。

第二篇 把边界建起来

第一篇做出了第一个插件，但协议很薄：三个函数、一个命令表，插件能做的事情有限，宿主也没有办法保护自己。这一篇补上四根柱子：契约、注册表、事件、隔离。

第 4 章把接口变成一份能检查、能版本化的合同。

第 5 章用注册表管理命令和扩展点，处理重名与卸载。

第 6 章让插件之间通过事件和钩子协作，而不必互相认识。

第 7 章把插件放进独立进程，让崩溃和死循环不再拖垮宿主。

第 4 章 接口就是合同

本章的完整代码和编译说明在 `codes/ch04/`。

4.1 第 3 章的插件还不会做事

第 3 章的插件能被宿主找到、加载、初始化、卸载，但它什么也做不了。

宿主知道这个插件叫 `hello`，知道什么时候该调用 `plugin_init`，但不知道插件里有什么功能。用户输入 `hello` 时，宿主没有函数可调。

要让插件真正提供命令，得完成一次信息交接：插件把“我有一个命令，名字叫 hello，执行时调用这个函数”告诉宿主；宿主记下来，等用户输入时再查。

问题在于，“告诉”这个动作本身需要一个函数。宿主得调用插件里的某个函数，插件才能把命令交出去。可第 3 章的三个函数里，没有哪个是用来接收命令的。

所以这一章要连续解决三件事：

1. 插件用什么形式描述一个命令
2. 插件通过什么函数把命令交给宿主
3. 宿主怎样把那个函数递给插件

三件事环环相扣，少一件都接不上。

4.2 先看一次完整的交接

在写代码之前，先把整个过程在脑子里过一遍。假设宿主已经加载了 `hello.dll`。

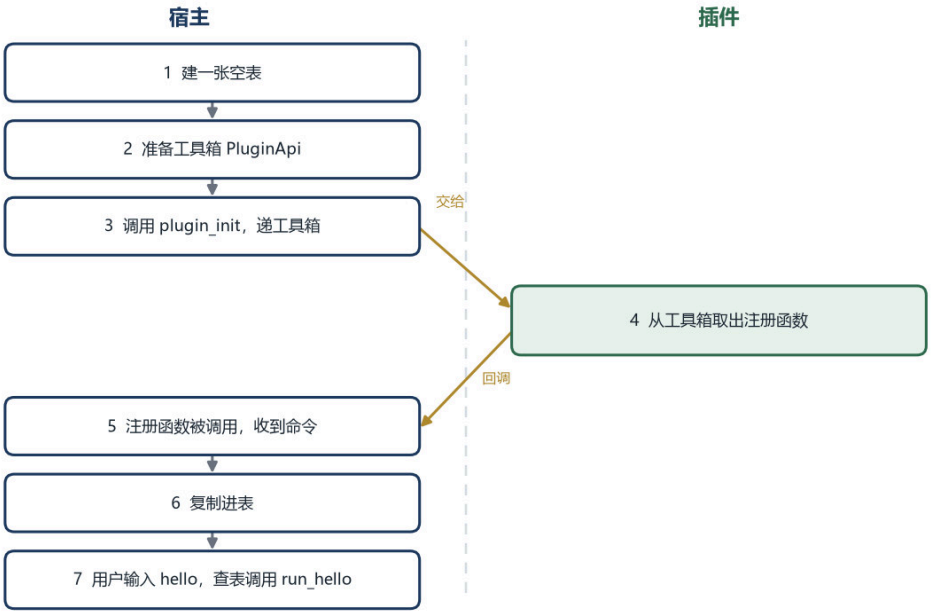


图 4-1　一条命令从插件交到宿主手里的七个步骤

这里有两次方向相反的传递。

第一次是宿主把**工具箱**递给插件。工具箱里装的不是数据，是函数地址。插件拿到之后，才能反过来调用宿主。

第二次是插件把**命令**递给宿主。命令里装的也是函数地址，指向插件自己的执行函数。

两次传递都只传“地址”和“简单的结构体”，不传 C++ 对象。为什么？下一节解释。

4.3 为什么不能直接传 C++ 对象

最自然的想法是：插件把一个 `std::string` 当命令名，把一个 `std::function` 当执行函数，打包成一个 C++ 类，交给宿主。

这个想法在同一个程序、同一套编译环境里通常没问题；一旦插件是别人用别的编译器做的，就会出事。

`std::string` 不是一个简单的字符数组。它内部通常有三个东西：一个指向字符数据的指针、一个表示长度的整数、一个表示容量的整数。不同标准库的实现，这三个东西的顺序、大小、甚至是否用“短字符串优化”都不一样。

Visual Studio 的 `std::string` 和 GCC 的 `std::string`，在内存里的样子不同。插件用 Visual Studio 编译，宿主用 GCC 编译，插件交过来一个 `std::string`，宿主按自己的理解去读，读到的长度可能是乱的，指针可能指向错误的位置。程序不会给你一个“类型不匹配”的提示，它会直接读到垃圾，然后崩溃。

`std::function` 更复杂。它内部可能存函数指针，也可能存一个对象，还可能有虚表。跨编译器传递，等于把两个不同世界的规则混在一起。

这就是跨动态库接口的第一条硬规则：

只传两边布局完全一致的东西。

哪些东西布局一致？整数、浮点数、指针、C 风格字符串（`const char*`）、只包含这些字段的结构体。它们的排列规则由平台和编译器共同遵守，Visual Studio 和 GCC 在同一个平台上通常一致。

所以命令不能用 C++ 类描述，要用一个“只有指针”的结构体描述。

4.4 用三个字段描述一个命令

```
typedef void (*CommandFn)(const char* args);

typedef struct PluginCommand {
    const char* name;
    const char* help;
    CommandFn run;
} PluginCommand;
```

先看 `CommandFn`。它定义了一种函数类型：接收一个 `const char*`，不返回值。以后凡是“执行命令”的函数，都长这样。

`const char*` 是“指向一串字符的指针”，字符内容不能被修改。用 C 风格字符串而不是 `std::string`，就是因为它的内存布局最简单：一串字节，末尾一个

`\0` 表示结束。两个编译器看起来完全一样。

`PluginCommand` 有三个字段。

`name` 是命令名，比如 `hello`。`help` 是一句说明，显示帮助时用。`run` 是执行函数的地址，类型是 `CommandFn`。

`typedef struct` 是 C 的写法，把“定义结构体”和“起一个类型名”合并成一句。写成 `struct PluginCommand { ... };` 再 `typedef struct PluginCommand PluginCommand;` 效果一样，只是啰嗦。

这个结构体里没有任何 C++ 对象，只有三个指针。三个指针的大小在同一平台上固定，字段顺序由声明决定。插件和宿主看到的是同一张内存图，交接才安全。

有一点要注意：`name` 和 `help` 指向的字符内容在插件里。宿主拿到指针后，不能假设它永远有效。如果插件被卸载，那块内存就没了。所以宿主收到命令后，应该把字符串复制一份，而不是只存指针。这一点在 4.12 节的宿主代码里会看到。

4.5 把命令交出去的那个函数

现在有了命令的格式，还需要一个函数，让插件把命令递给宿主。

```
typedef void (*RegisterCommandFn)(const PluginCommand* cmd);
```

读这种写法有个窍门：从名字往外读。

`(*RegisterCommandFn)` 表示 `RegisterCommandFn` 是一个指针。右边的 `(const PluginCommand*)` 表示它指向的函数接收一个参数，参数是指向 `PluginCommand` 的指针。左边的 `void` 表示那个函数不返回值。

合起来：`RegisterCommandFn` 是“指向一个注册函数”的指针，那个注册函数接收一条命令，不返回东西。

这个函数由宿主实现，插件调用。插件在自己的 `plugin_init` 里，把命令一条条传给它。

就像去银行办业务。窗口是银行开的，你走过去，把表格递进窗口。窗口不在你手里，但你能用它。注册函数就是那个窗口。

4.6 工具箱：PluginApi

宿主不能只递一个注册函数。以后可能还要提供日志、读文件、发通知。每加一项能力就改一次 `plugin_init` 的参数，签名会越来越长，旧插件也会全部失效。

更好的做法是把所有能力放进一个结构体，只传这个结构体的指针。以后加能力，往结构体末尾加字段就行。

```
typedef struct PluginApi {
    uint32_t size;
    uint32_t version;
    RegisterCommandFn register_command;
    void (*log_info)(const char* message);
} PluginApi;
```

四个字段，分两类。

前两个是“说明书”。`size` 记录这个结构体有多大，`version` 记录契约版本。

后两个是“能力”。`register_command` 是刚才定义的注册函数；`log_info` 是日志函数，插件想输出信息时调用它，而不是直接往屏幕打印。这样宿主可以统一控制日志格式、写到文件、按插件过滤。

`uint32_t` 是固定 4 字节的无符号整数，来自 `<cstdint>`。用它而不是 `int`，是因为 `int` 的大小在不同平台上可能不同，而 `uint32_t` 永远是 4 字节。

这四个字段在内存里依次排列，宿主填好之后，把整个结构体的地址交给插件。



图 4-2　PluginApi 的四个字段依次排列，宿主填好后把地址交给插件

4.7 为什么结构体开头要放一个尺码

`size` 和 `version` 不是装饰。它们解决的是“插件和宿主不是同一天编译”的问题。

设想一个场景。宿主 1.0 发布时，`PluginApi` 只有三个字段：`size`、`version`、`register_command`。半年后契约升级到 1.1，在末尾加了 `log_info`。这时会出现两种组合。

第一种，新插件装在旧宿主上。插件按 1.1 写，想用 `log_info`。它拿到宿主给的 `PluginApi`，先读 `size`，发现只覆盖到 `register_command`，就知道“这个宿主没有日志能力”，于是走不用日志的路径，而不是硬读一个不存在的字段。

第二种，旧插件装在新宿主上。插件按 1.0 写，只认识前三个字段。宿主给的 `size` 更大，插件不关心后面多出来的内容，照常工作。

`size` 让双方都能安全地只使用自己认识的那一段。

宿主创建 `PluginApi` 时，把 `size` 填成 `sizeof(PluginApi)`，也就是“我这个结构体有多少字节”。插件拿到指针，先读 `size`，和自己的预期比一比：

- 如果 `size` 够大，说明宿主提供了自己要用的字段，可以用
- 如果 `size` 太小，说明宿主是旧版，没有这个字段，插件就退回旧路径

这个比较不需要知道对方的结构体具体长什么样，只需要知道“有没有这么多字节”。就像收快递先看箱子上的件数，数一数够不够，再决定拆不拆。

这就像搬家时箱子上的清单。清单上写着“共 12 件”，你打开箱子先数一遍，数出来只有 10 件，就知道少了两件，不会假装没看见继续往下搬。

`size` 只能保证“字段够不够”，不能保证“字段的意思没变”。所以还需要 `version`。

4.8 version 管的是意思有没有变

假设宿主没有加字段，但把 `register_command` 的行为改了：以前是“收到命令立刻复制一份”，现在改成“只记指针，等插件卸载时再复制”。字段数量没变，`size` 也一样，插件却会因此出问题。

这种变化，`size` 检测不出来，只能靠版本号。

规则很简单：不兼容的改动，版本号加一；兼容的改动，不加。

什么叫兼容？往结构体末尾加字段是兼容的，旧插件不读新字段，新宿主也能用旧插件。放宽参数范围、修正一个明显是 bug 的行为，通常也算兼容。

什么叫不兼容？改变已有字段的含义、删除字段、改变函数签名、改变调用顺序。这些都要升版本。

插件在初始化时检查：

```
if (api->version != PLUGIN_API_VERSION) {
    return 2;  // 版本不匹配，拒绝加载
}
```

这里用“不相等就拒绝”，是最简单的策略。真实系统可能同时支持几个版本，让旧插件走一条兼容路径，第 10 章会讲。

4.9 只给能力，不给内部数据

`PluginApi` 里放的都是函数，没有一个字段是宿主内部的数据结构。这不是巧合，是刻意的设计。

假设宿主有一个文档对象，里面有路径、内容、修改时间、撤销历史。如果为了省事，把 `Document*` 直接放进 `PluginApi`，插件就能读到所有字段，也能改它们。宿主以后想调整这个结构，所有插件都会坏。

正确的做法是只暴露操作。插件想读内容，给它一个 `read_document` 函数；想改内容，给它一个 `write_document` 函数。它不需要知道文档对象长什么样。

自动售货机就是这个道理。你投币、按按钮、拿饮料，不需要知道机器里的货道怎么排、弹簧怎么转。机器也不允许你伸手进去自己拿。按钮是能力，货道是内部数据。

`PluginApi` 就是那排按钮。以后每加一项能力，就往结构体末尾加一个函数指针，而不是加一个数据指针。

4.10 完整的接口头文件

把前面的定义合起来，得到这一章的最终版本：

```
// plugin_api.h
#pragma once

#include <cstdint>

#ifdef _WIN32
#define PLUGIN_EXPORT __declspec(dllexport)
#else
#define PLUGIN_EXPORT __attribute__((visibility("default")))
#endif

#define PLUGIN_API_VERSION 1

extern "C" {

typedef void (*CommandFn)(const char* args);

typedef struct PluginCommand {
    const char* name;
    const char* help;
```

```
    CommandFn run;
} PluginCommand;

typedef void (*RegisterCommandFn)(const PluginCommand* cmd);

typedef struct PluginApi {
    uint32_t size;
    uint32_t version;
    RegisterCommandFn register_command;
    void (*log_info)(const char* message);
} PluginApi;

PLUGIN_EXPORT const char* plugin_name();
PLUGIN_EXPORT int plugin_init(const PluginApi* api);
PLUGIN_EXPORT void plugin_shutdown();

}
```

`PLUGIN_EXPORT` 和第 3 章一样，Windows 上展开成导出标记，Linux 上展开成可见性标记。

`PLUGIN_API_VERSION` 是当前契约版本。插件和宿主都用这个宏，升级时只改一处。

`extern "C"` 让三个导出函数按 C 的规则命名，避免第 3 章说的名字修饰问题。结构体定义放在 `extern "C"` 里面，是为了让整份头文件在 C 和 C++ 编译器里都能用。

4.11 插件这边怎么写

插件要做三件事：定义执行函数、把它们放进一个静态数组、在初始化时逐条注册。

```
#include "plugin_api.h"
#include <cstdio>

static const PluginApi* g_api = nullptr;

static void run_hello(const char* args) {
    if (g_api && g_api->log_info) {
        g_api->log_info("hello 命令被执行");
    }
    std::printf("你好，世界（参数：%s）\n", args ? args : "");
    std::fflush(stdout);
}

static const PluginCommand kCommands[] = {
```

```
    { "hello", "打印一句问候", run_hello },
};

extern "C" PLUGIN_EXPORT const char* plugin_name() {
    return "hello";
}

extern "C" PLUGIN_EXPORT int plugin_init(const PluginApi* api) {
    if (!api) return 1;
    if (api->version != PLUGIN_API_VERSION) return 2;

    g_api = api;
    for (const auto& cmd : kCommands) {
        api->register_command(&cmd);
    }
    return 0;
}

extern "C" PLUGIN_EXPORT void plugin_shutdown() {
    g_api = nullptr;
}
```

`g_api` 保存宿主递过来的工具箱。命令执行时如果需要日志，就通过它调用。`static` 让这个变量只在本文件可见，不会和别的插件重名。

`kCommands` 是一个静态数组，里面放这个插件提供的所有命令。现在只有一条。加命令时，往数组里加一行就行。

`kCommands` 和里面的字符串都是静态存储，生命周期和程序一样长，所以把指针交给宿主是安全的，不会因为函数返回就失效。

`plugin_init` 的顺序很重要：先检查指针，再检查版本，然后保存工具箱，最后注册命令。任何一步失败，都返回非零，宿主会跳过这个插件。

`plugin_shutdown` 把 `g_api` 清空，防止卸载后还有人通过它调用宿主。

4.12 宿主这边怎么写

宿主需要做四件事：建表、实现注册函数和日志函数、填好工具箱、调用插件的初始化。

先看表的实现：

```
#include <cstdio>
#include <map>
#include <string>

struct CommandEntry {
    std::string name;
    std::string help;
    CommandFn run;
};

static std::map<std::string, CommandEntry> g_commands;

static void host_register_command(const PluginCommand* cmd) {
    if (!cmd || !cmd->name || !cmd->run) return;

    CommandEntry entry;
    entry.name = cmd->name;
    entry.help = cmd->help ? cmd->help : "";
    entry.run = cmd->run;
    g_commands[entry.name] = entry;

    std::printf("registered command: %s\n", cmd->name);
}

static void host_log_info(const char* message) {
    std::printf("[host] %s\n", message ? message : "");
}
```

`std::map` 是标准库的键值表，按名字存命令，查找很快。`g_commands[entry.name] = entry` 表示把这条命令放到这个名字下面。

注意 `entry.name = cmd->name` 这一行。它把 `const char*` 赋给 `std::string`，`std::string` 会自己复制一份字符内容。这样即使插件后来被卸载，宿主表里的名字仍然有效。

这里用 `std::string` 和 `std::map` 是安全的，因为它们只在宿主内部使用，不跨动态库传递。跨边界传的是 `const char*`，进了宿主之后才变成 `std::string`。

然后填工具箱并调用：

```
PluginApi api{};
api.size = sizeof(PluginApi);
api.version = PLUGIN_API_VERSION;
api.register_command = host_register_command;
api.log_info = host_log_info;

auto init = (int(*)(const PluginApi*))get_sym(h, "plugin_init");
if (!init || init(&api) != 0) {
```

```
        std::printf("plugin init failed\n");
        close_lib(h);
        continue;
    }
```

`PluginApi api{}` 把所有字段先清零，避免忘记赋值时留下垃圾。

`api.size = sizeof(PluginApi)` 填的是宿主这个结构体的大小。`api.version` 填契约版本。

两个函数指针分别指向宿主实现的注册函数和日志函数。插件拿到结构体后，调用 `api->register_command(&cmd)`，实际执行的是宿主的 `host_register_command`。

这就是 4.2 节那张图里“宿主把工具箱递过去，插件反过来调用宿主”的具体实现。

4.13 这一章定下了什么

到这里，插件和宿主之间有了一份真正的合同。合同包括五条：

- 插件导出三个固定名字的函数
- 函数用 `extern "C"`，避免名字修饰
- 命令用只含指针的结构体描述，不用 C++ 对象
- 宿主通过 `PluginApi` 提供能力，插件只能调用这些能力
- `PluginApi` 带 `size` 和 `version`，双方可以检查兼容性

这份合同还很粗糙。命令注册进去之后，怎么执行？用户输入 `hello`，宿主怎么从表里找到对应的函数？两个插件注册同名命令怎么办？插件卸载时，它注册的命令怎么撤掉？

下一章做注册表，把这些事一件件处理掉。

第 5 章 注册表与扩展点

本章的完整代码和编译说明在 `codes/ch05/`。

5.1 命令注册进去之后

第 4 章的宿主已经能把插件交上来的命令存进一张表，但还没有用它。这一章把表用起来，让用户输入 `hello` 时真的调用到插件里的函数。

先明确表要存什么。

```
struct Command {
    std::string name;
    std::string help;
    CommandFn run;
    std::string owner;
};
```

前三个字段和第 4 章一样。第四个 `owner` 是新增的，记录这条命令是哪个插件注册的。为什么需要它？因为插件可能被卸载，卸载时要把它注册的东西全部撤掉。没有 `owner`，就不知道哪些条目属于它。

`owner` 就是插件名，来自 `plugin_name()`。宿主在调用 `plugin_init` 之前先拿到名字，注册命令时自动填进去。插件自己不需要传，也没法冒充别人。

5.2 一张能查、能删的表

用 `std::map` 存命令，键是命令名。

```
class CommandTable {
public:
    bool add(const Command& cmd) {
        if (cmd.name.empty() || !cmd.run) return false;
        if (commands_.count(cmd.name) > 0) return false;
        commands_[cmd.name] = cmd;
        return true;
```

```
    }

    const Command* find(const std::string& name) const {
        auto it = commands_.find(name);
        if (it == commands_.end()) return nullptr;
        return &it->second;
    }

    void remove_owner(const std::string& owner) {
        for (auto it = commands_.begin(); it != commands_.end(); ) {
            if (it->second.owner == owner) {
                it = commands_.erase(it);
            } else {
                ++it;
            }
        }
    }

    void list() const {
        for (const auto& pair : commands_) {
            std::printf("  %-12s %s\n", pair.first.c_str(),
pair.second.help.c_str());
        }
    }

private:
    std::map<std::string, Command> commands_;
};
```

`add` 做三件事：检查名字和执行函数不为空、检查有没有重名、存进表。重名时返回 `false`，由调用方决定怎么处理。

`find` 按名字查。找不到返回空指针。返回的是指针，不是副本，避免复制整个结构体。

`remove_owner` 遍历整张表，把属于某个插件的条目删掉。`erase` 会返回下一个位置，所以删除时用 `it = commands_.erase(it)`，不能直接 `++it`，否则迭代器失效。

`list` 打印所有命令，`%-12s` 表示左对齐、占 12 个字符宽度，让输出整齐。

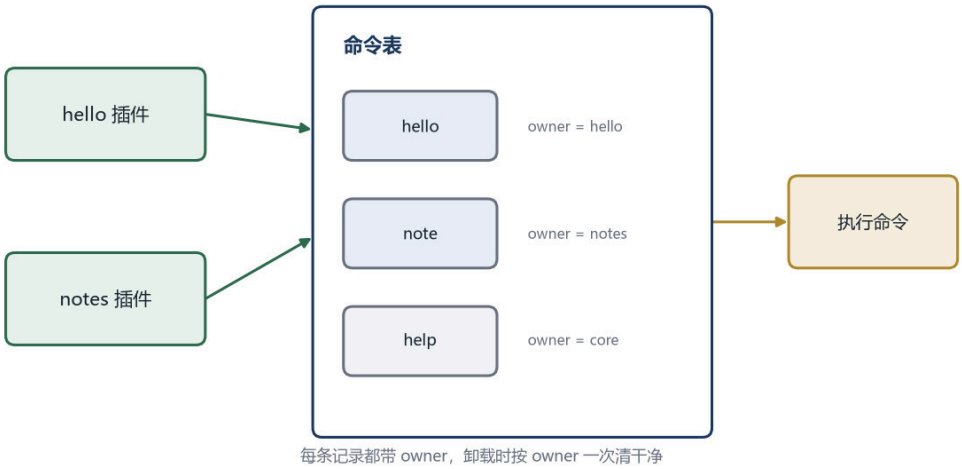


图 5-1　命令表里的每一条都记录着注册者

5.3 执行一条命令

主循环拿到用户输入后，先按空格切开，第一个词是命令名，剩下的都是参数。

```
static std::string trim(const std::string& s) {
    size_t begin = s.find_first_not_of(" \t\r\n");
    if (begin == std::string::npos) return "";
    size_t end = s.find_last_not_of(" \t\r\n");
    return s.substr(begin, end - begin + 1);
}

static void run_line(CommandTable& table, const std::string& line) {
    std::string text = trim(line);
    if (text.empty()) return;

    size_t space = text.find(' ');
    std::string name = (space == std::string::npos) ? text : text.substr(0, space);
    std::string args = (space == std::string::npos) ? "" : trim(text.substr(space +
1));

    const Command* cmd = table.find(name);
    if (!cmd) {
        std::printf("未知命令：%s（输入 help 查看）\n", name.c_str());
        return;
    }
    cmd->run(args.c_str());
}
```

`find_first_not_of` 找第一个不是空白的位置，`find_last_not_of` 找最后一个不是空白的位置。两者一夹，就得到去掉首尾空白后的范围。`npos` 表示“没找到”。

`text.find(' ')` 找第一个空格。找不到说明整行都是命令名，参数为空；找到了就切成两段。

`args.c_str()` 把 `std::string` 转成 C 风格字符串，因为命令函数接收的是 `const char*`。这个指针只在 `run` 执行期间有效，插件不能把它存起来以后再用。

5.4 重名怎么办

两个插件都注册 `hello`，谁赢？

有三种常见策略。

拒绝后来者。 第一个注册成功，第二个失败。简单、可预测，但用户不知道为什么自己的插件没生效。

覆盖前者。 后来的替换先前的。方便调试，但容易让行为变得不可预测，尤其是加载顺序不固定的时候。

报错并禁用。 检测到冲突时，把两个插件都标记为冲突，提示用户处理。最安全，但对用户要求高。

这一章选第一种，并在日志里说清楚是谁和谁冲突。真实系统通常会允许插件在清单里声明优先级，高优先级的先注册。

```
bool add(const Command& cmd) {
    auto it = commands_.find(cmd.name);
    if (it != commands_.end()) {
        std::printf("命令冲突：%s 已被 %s 注册，%s 注册失败\n",
                    cmd.name.c_str(), it->second.owner.c_str(), cmd.owner.c_str());
        return false;
    }
    commands_[cmd.name] = cmd;
    return true;
}
```

冲突信息里带上了双方的名字，用户一看就知道该禁用哪个。

5.5 卸载时清理

插件被禁用或者程序退出时，要把它注册的命令撤掉。

```
void unload_plugin(const std::string& owner, CommandTable& table) {
    table.remove_owner(owner);
    std::printf("已卸载插件：%s\n", owner.c_str());
}
```

顺序很重要。先调用插件的 `plugin_shutdown`，让它自己清理内部状态；再把它注册的命令从表里删掉。反过来的话，插件在清理时可能还想访问自己的命令，却发现已经没了。

宿主加载每个插件时，会记下它的名字和 `plugin_shutdown` 的地址。用一个很小的结构体保存：

```
struct LoadedPlugin {
    std::string name;      // 插件名，用来按 owner 清理
    void (*shutdown)();    // 指向插件导出的 plugin_shutdown
};
```

退出时按相反顺序逐个停用：

```
void shutdown_all(std::vector<LoadedPlugin>& plugins, CommandTable& table) {
    for (auto it = plugins.rbegin(); it != plugins.rend(); ++it) {
        if (it->shutdown) it->shutdown();
        table.remove_owner(it->name);
    }
    plugins.clear();
}
```

`rbegin` 和 `rend` 是反向遍历，从最后一个插件开始卸载。为什么倒着来？因为后面的插件可能依赖前面的插件提供的服务，先卸载后面的更安全。

5.6 命令表不够用了

命令表解决了一个具体问题：插件怎么把“一条命令”交给宿主。但一个大型软件需要插件参与的地方远不止命令。

窗口顶部有菜单，菜单里有“文件”“编辑”“帮助”，每一项都可能由插件添加。工具栏上有一排按钮，每个按钮也可能来自插件。用户双击一个 `.md` 文件时，宿主需要问一圈“谁能打开这个文件”。设置界面里，每个插件可能想加一页自己的配置。

这些位置有一个共同点：宿主先留好一个“坑”，插件往坑里填东西，宿主在合适的时候把坑里的东西取出来用。

命令是一个坑，菜单项是另一个坑，文件处理器是第三个坑。它们的名字不同，数据结构不同，取用的时机也不同，但“留坑、填坑、取用”这个模式是一样的。

如果每类坑都从头写一遍增删查改，代码会重复得很难看，而且每一份都可能漏掉“卸载时清理”。把这些共性抽出来，就是**扩展点**。

5.7 扩展点的三个要素

光说“扩展点是一个坑”还不够。要让宿主和插件都能准确理解这个坑，需要三样东西。

第一样：一个稳定的名字

宿主先声明：“我这里有一个位置，叫 `menu.file`。”插件看到这个名字，就知道该往哪里填。

名字要稳定，因为插件会把它写进代码里。今天叫 `menu.file`，明天改成 `file_menu`，所有插件都得改。名字还要有层级，`menu.file` 表示“文件菜单”，`menu.edit` 表示“编辑菜单”，一眼能看出位置。

命令扩展点的名字是 `command`。它没有更细的层级，因为命令之间是平级的，靠命令名区分。

第二样：一份数据形状

光有名字，宿主不知道插件会填什么进来。所以还要约定填进来的东西长什么样。

命令的形状是：

```
struct Command {
    std::string name;
    std::string help;
    CommandFn run;
    std::string owner;
};
```

菜单项的形状可能是：

```
struct MenuItem {
    std::string title;
    std::string shortcut;
    void (*on_click)();
    std::string owner;
};
```

两者都有名字和执行函数，但菜单项还多了标题和快捷键。宿主拿到菜单项，知道怎么画；拿到命令，知道怎么执行。形状不对，宿主就没法用。

第三样：一个调用时机

有了名字和形状，还要说清楚“宿主什么时候把这些东西取出来”。

命令的时机是：用户输入一行文字，宿主按第一个词去表里查，查到就调用。

菜单项的时机是：用户点开某个菜单，宿主把这个菜单下所有项取出来，按顺序画在屏幕上。

文件处理器的时机是：用户打开一个宿主不认识的文件，宿主问每个处理器“你能处理 `.md` 吗”，第一个说能的就是它。

三样东西缺一不可。只有名字，宿主不知道填什么；只有形状，不知道往哪填；只有时机，不知道取出来的是什么。

可以用小区公告板来理解。物业说“三号楼门口有一块公告板”，这是名字；“只能贴 A4 纸，写上标题和联系方式”，这是形状；“每天早上八点统一收走”，这是时机。三者齐了，住户才知道该怎么做。

5.8 三种存法，各自的问题

命令和菜单项的形状不同，C++ 是强类型语言，一个 `std::map<std::string, Command>` 装不下 `MenuItem`。怎么把不同形状的东西放进同一套管理逻辑，有三种常见做法。

做法一：每种类型一张表

最直接的想法是给每类扩展点各写一张表：

```
std::map<std::string, Command> commands;
std::map<std::string, MenuItem> menus;

bool add_command(const Command& cmd) { /* 检查重名、插入 */ }
bool add_menu(const MenuItem& item) { /* 检查重名、插入 */ }
```

优点是类型安全。往 `commands` 里塞一个 `MenuItem`，编译器立刻报错，不会等到运行时才出问题。

缺点是重复。`add_command` 和 `add_menu` 的逻辑一模一样，只是类型不同；`find`、`remove_owner`、`list` 也要各写两遍。扩展点增加到十种，就有十套几乎相同的代码。更麻烦的是，将来修一个 bug，比如“卸载时忘了清空”，要记得在十个地方都改一遍。

做法二：用 void* 装一切

`void*` 是“只知道地址、不知道类型”的指针。任何指针都能转成 `void*`，取出来时再转回去。

```
struct Entry {
    std::string owner;
    void* value;
};
```

```
std::map<std::string, Entry> items;
```

存进去：

```
Command cmd{ "hello", "打印问候", run_hello, "hello" };
items["hello"] = { "hello", &cmd };
```

取出来：

```
auto* cmd = static_cast<Command*>(items["hello"].value);
cmd->run("");
```

看起来一张表就装下了所有类型，代码也不用重复。问题出在“转回去”这一步。

`static_cast` 不做运行时检查。如果你记错了类型，写成 `static_cast<MenuItem*>`，编译器不会拦你。程序会把一块 `Command` 的内存当成 `MenuItem` 来读，字段位置全错，读出来的标题可能是一个函数地址，调用时直接崩溃。

这种错误很难查，因为崩溃的地方和写错的地方可能隔着几百行。表越大、类型越多，记错的概率越高。

`void*` 还有第二个问题：它不知道对象该怎么销毁。表里存的是指针，清理时是 `delete`、`free`，还是什么都不做？不同类型的扩展可能需要不同的清理方式，一张 `void*` 表回答不了。

做法三：基类加虚函数

面向对象的标准做法是让所有扩展继承同一个基类：

```
struct Extension {
    virtual ~Extension() = default;
};

struct Command : Extension {
    std::string name;
    CommandFn run;
};

struct MenuItem : Extension {
    std::string title;
```

```
    void (*on_click)();
};

std::map<std::string, std::unique_ptr<Extension>> items;
```

`std::unique_ptr<Extension>` 是一种会自动释放对象的指针，离开作用域时自动 `delete` 指向的对象，不用手写 `delete`。`<Extension>` 表示它指向的东西是一个 `Extension`。

基类里的 `virtual ~Extension() = default` 是虚析构函数，保证通过基类指针删除对象时，能调用到子类自己的析构函数。

存的时候，`Command` 和 `MenuItem` 都能转成 `Extension*`，一张表装得下。取的时候用 `dynamic_cast<Command*>` 转回具体类型，转错了会返回空指针，比 `void*` 安全。

但在插件系统里，这个方案有一个致命问题：**虚表不能跨编译器**。

带虚函数的对象，内存开头有一个指针，指向“这个类有哪些虚函数”的表，叫虚表。虚表的布局由编译器决定，Visual Studio 和 GCC 排法不同，同一编译器的不同版本也可能不同。插件里创建的对象，宿主按自己的理解去读虚表，调用的可能是错误的函数。

还有内存分配的问题。插件里 `new` 出来的对象，如果在宿主里 `delete`，两边的堆管理器可能不是同一个，轻则泄漏，重则崩溃。要解决，得约定“谁分配谁释放”，或者所有对象都由宿主分配，复杂度立刻上去了。

所以跨动态库的接口，通常避开虚函数和 C++ 对象，这也是第 4 章只传简单结构体的原因。

这一章的选择

三种做法比较下来：

做法	类型安全	代码重复	跨编译器
每种类型一张表	安全	多	安全
void* 一张表	不安全	少	安全但易错
基类加虚函数	较安全	少	危险

这一章选第一种。类型安全和跨编译器安全比少写几行代码重要，尤其是插件可能来自不同编译器的时候。

重复的问题，用下一节的模板解决。

5.9 用模板消除重复

每种类型一张表，逻辑一样，只是类型不同。C++ 的**模板**就是为这种情况准备的。

模板的意思是“把类型当成参数”。写一个类，里面用到一个暂时不确定的类型 `T`，等真正使用时再填。

```
template <typename T>
class ExtensionPoint {
public:
    bool add(const std::string& key, const std::string& owner, const T& value) {
        if (items_.count(key) > 0) return false;
        items_[key] = { owner, value };
        return true;
    }

    const T* find(const std::string& key) const {
        auto it = items_.find(key);
        return it == items_.end() ? nullptr : &it->second.value;
    }

    void remove_owner(const std::string& owner) {
        for (auto it = items_.begin(); it != items_.end(); ) {
            if (it->second.owner == owner) it = items_.erase(it);
            else ++it;
        }
    }

private:
    struct Entry {
        std::string owner;
```

```
        T value;
    };
    std::map<std::string, Entry> items_;
};
```

`template <typename T>` 声明了一个类型参数 `T`。类里面的 `T` 暂时是个占位符。

真正使用时这样写：

```
ExtensionPoint<Command> commands;
ExtensionPoint<MenuItem> menus;
```

编译器看到第一行，用 `Command` 替换所有 `T`，生成一份“命令版”的类；看到第二行，用 `MenuItem` 替换，生成一份“菜单版”的类。两份代码逻辑一样，但类型各自独立，往 `ExtensionPoint<Command>` 里塞 `MenuItem` 照样会编译报错。

这就像同一个表格模板，打印一份“学生登记表”，再打印一份“图书登记表”。表格的格子布局一样，但一份填姓名学号，一份填书名编号，不能混着填。

模板只在编译时展开，不会跨动态库传递。插件通过 C 函数把 `PluginCommand` 交给宿主，宿主再存进自己的 `ExtensionPoint<Command>`。边界上仍然是第 4 章那些简单结构体，模板只在宿主内部使用。

5.10 三种扩展点形态

扩展点按“取出来之后怎么用”，可以分成三类。这三类的区别不在数据结构，而在调用方式。

列表型：各用各的

菜单项是典型的列表型。宿主打开“文件”菜单时，把这个位置下的所有菜单项取出来，按顺序画在屏幕上。第一项和第二项之间没有数据传递，各自独立。

工具栏按钮、设置页、状态栏文字，都属于这一类。宿主需要“全部”，顺序可能由优先级决定，但每一项的处理互不影响。

管道型：前一个的输出是后一个的输入

文本转换是典型的管道型。假设宿主支持 Markdown 预览，渲染过程分成几步：

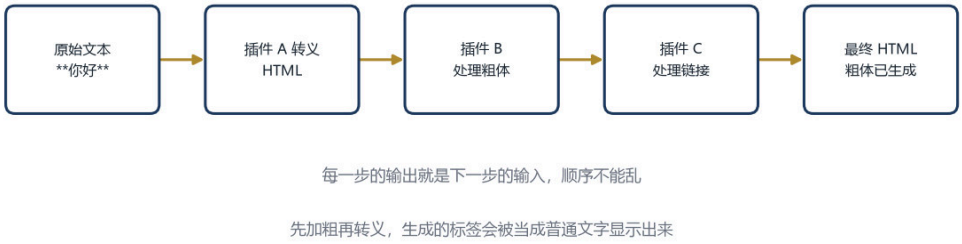


图 5-2　三个插件串成一条管道，前一个的输出是后一个的输入

每一步接收一段文本，返回一段文本。插件 B 拿到的输入，正是插件 A 的输出；插件 C 拿到的，又是插件 B 的输出。三个插件串成一条流水线，所以叫管道型。

图片滤镜也是同样的结构：

每个滤镜接收一张图，返回一张图。去噪的输出就是调色的输入，调色的输出再交给锐化，最后得到结果。

管道型的重点是**顺序**。上面三步如果换成“先处理粗体，再转义”，插件 B 生成的 `<b>` 会被插件 A 当成 HTML 标签转义成 `<b>`，页面上就会直接显示 `<b>` 这几个字符，而不是粗体。顺序错了，结果就错了。所以管道型扩展点必须把顺序写进契约，不能靠注册顺序碰运气。

管道型还要定义“某一环失败怎么办”。常见有两种：跳过这一环，把输入原样传给下一个；或者中止整个管道，返回错误。选哪种要看业务，但必须统一。

单点型：同一时间只能有一个

主题是典型的单点型。用户可以选浅色或深色，但同一时间只能生效一个。两个插件都提供主题时，宿主不能把两个都画上，必须选一个。

默认存储后端也是单点型。数据存在本地文件还是云端，只能选一条路。

单点型的冲突策略通常是“按优先级选一个”，或者“让用户在设置里选”。它和列表型的区别是：列表型可以共存，单点型必须互斥。

三类对比

形态	取出来怎么用	典型例子	冲突处理
列表型	全部取出，逐个展示或调用	菜单项、工具栏按钮	可以共存，按顺序
管道型	串成一条链，前一个输出给后一个	文本转换、图片滤镜	必须定义顺序
单点型	只取一个生效	主题、默认存储	互斥，按优先级或用户选

设计扩展点时，先想清楚属于哪一类，再决定数据结构、冲突策略和清理方式。命令表是列表型的一个特例：它按名字索引，每个名字一条，取出来执行一条。

5.11 把这一章串起来

这一章从“命令注册进去之后”讲到“扩展点怎么设计”，中间有七步。单独看每一节都懂，合起来容易乱，所以先把七步连成一条线：

1. 宿主建一张空表（5.2）
2. 插件加载时拿到 `PluginApi`（第 4 章）
3. 插件注册命令，宿主填上 `owner`（5.1）
4. 用户输入 `hello 世界`
5. 宿主按名字查表，找到 `Command`（5.3）

6. 调用 `run` 函数指针，执行插件的代码（5.3）
7. 退出时按 `owner` 把命令清理掉（5.5）



图 5-3　一条命令从注册到清理经过的七步

前六步是命令表的全部工作。第七步之后，把“命令”换成“菜单项”“文件处理器”，把表的类型换成对应的结构，就是 5.6 到 5.10 讲的扩展点。

把这些步骤写成代码，主程序大概是这样：

```
static CommandTable g_table;

static void run_help(const char*) {
    g_table.list();
}

// 扫描 plugins 目录、加载动态库、调用 plugin_init
// 实现在 codes/ch05/host.cpp，对应第 3 章和 5.1 节
static void load_all_plugins();

// 调用 plugin_shutdown，再按 owner 清理命令，对应 5.5 节
static void unload_all_plugins();

int main() {
    // 创建宿主自己的 help 命令，并加入到命令表里
    Command help;
    help.name = "help";
    help.help = "列出所有命令";
    help.owner = "core";
```

```
    help.run = run_help;
    g_table.add(help);

    load_all_plugins();

    std::string line;
    while (true) {
        std::printf("> ");
        std::fflush(stdout);
        if (!std::getline(std::cin, line)) break;
        if (line == "quit") break;
        run_line(g_table, line);
    }

    unload_all_plugins();
    return 0;
}
```

对照前面的小节看：

- `g_table` 是 5.2 定义的那张命令表
- `run_help` 是宿主自己的命令，它需要通过 `g_table` 访问命令表，所以表放在文件级
- `load_all_plugins` 把第 3 章的“扫描目录、加载动态库、取符号”和第 4 章的“传 `PluginApi`、调用 `plugin_init`”合在一起，插件的命令就在这一步进表
- `run_line` 是 5.3 的“切分输入、查表、调用”
- `unload_all_plugins` 是 5.5 的“先调用 `plugin_shutdown`，再按 owner 删命令”

完整可编译的版本在 `codes/ch05/`，运行后输入 `help` 能看到命令列表，输入 `hello` 会调用插件的函数，输入 `quit` 退出。

这一章的主线可以压成一句话：**宿主留一张表，插件往里填，宿主按名字取用，卸载时按主人清理。** 扩展点是把“表”和“填的东西”抽象出来，让同一套逻辑服务菜单、文件处理器和设置页。

5.12 还差什么

命令能注册、能执行、能清理了。但插件之间还不能互相通知。

假设一个插件负责保存文件，另一个插件想在文件保存后更新索引。保存插件不知道索引插件的存在，索引插件也不知道保存插件什么时候保存。它们需要一个中间人。

这个中间人叫事件总线。下一章讲它，以及和它很像、但用途不同的钩子。

第 6 章 事件与钩子

本章的完整代码和编译说明在 `codes/ch06/`。

6.1 一次保存涉及哪些角色

假设有一个笔记应用。用户点保存按钮，或者输入 `save` 命令。

这个应用里有三部分：

- **宿主**：持有当前文档（内容、路径），提供 `save` 命令，负责真正把内容写到磁盘。
- **索引插件**：订阅“文件已保存”事件，更新搜索索引。
- **云插件**：订阅同一个事件，把新版本上传。

一次保存的流程是这样的：

1. 宿主收到 `save` 命令，取出当前文档内容
2. 宿主调用保存前的钩子，插件可以修改准备写出去的副本，也可以阻止保存
3. 宿主写文件
4. 宿主发布 `file.saved` 事件
5. 索引插件和云插件收到事件，各自处理

这里有一个容易误解的地方，需要先说清楚：**保存这个动作由宿主完成，不是由插件完成。**

你可能会想：插件不是应该负责具体功能吗，为什么保存不交给插件？

有三个原因。

第一，写文件不只是“把字符串写进文件”。要处理路径是否合法、目录是否存在、磁盘满不满、写到一半失败怎么回滚、要不要先写临时文件再改名。这些逻辑如果每个插件都写一遍，迟早不一致。

第二，权限要统一。插件能写哪些目录，应该由宿主控制。如果保存由插件做，插件就拿到了完整的文件路径，可以写到任何地方。

第三，顺序要统一。保存前要执行钩子，保存后要发事件。这套流程如果由每个插件自己实现，顺序和时机都会不同，订阅者拿到的数据也就不可靠。

所以这一章的分工是：**宿主负责“做”，插件负责“改”和“听”**。钩子让插件改内容，事件让插件知道结果。

另一种做法是插件自己保存。比如“导出 PDF”插件，它有自己的格式和写文件逻辑，宿主不需要知道 PDF 怎么生成。这种插件可以自己写文件，写完之后通过 `PluginApi` 上的 `emit_event` 告诉宿主“我完成了”，宿主再把事件转给总线。这种模式适合“插件提供的是一项独立服务”，不适合“保存当前文档”这种宿主核心能力。这一章不实现 `emit_event`，只说明有这种模式，免得读者以为所有事件都必须由宿主发布。

这一章用第一种模式，因为笔记应用的文档由宿主管理。后面说到“保存”，都指宿主执行保存。

6.2 事件长什么样

一个事件至少要有两样东西：类型和数据。

类型是一个字符串，说明“发生了什么”。这里用 `file.saved`，读作“文件已保存”。类型要稳定，因为插件会把它写进代码里，今天叫 `file.saved`，明天改成 `saved`，所有插件都得改。

数据是这件事的细节。这里用 `path=/notes/a.md`，表示保存的是哪个文件。数据也用一个字符串，格式是“键=值”，多个键值对可以用分号隔开。这一章不引入 JSON 库，因为重点是机制，不是格式解析；真实系统可以用 JSON，但解析器要放在宿主里，插件只负责填字符串。

两者合起来，就是一次通知。它包含两样东西：

- 类型：`file.saved`
- 数据：`path=/notes/a.md`

宿主要把这条通知送给订阅者，最直接的方式是调用一个函数，把类型和数据作为参数传进去。这个函数的类型定义如下：

```
typedef void (*EventListenerFn)(const char* type, const char* payload);
```

这行代码需要逐段读。从名字 `EventListenerFn` 往外看：

- 最外层是 `void`，表示这个函数不返回任何值
- `(*EventListenerFn)` 表示 `EventListenerFn` 是一个指针
- 右边的 `(const char* type, const char* payload)` 表示它指向的函数接收两个参数，都是字符串

合起来：`EventListenerFn` 是“指向一个事件处理函数”的指针，那个函数接收类型和数据，不返回东西。

这里要区分三个词，后面才不会乱。

事件处理函数是这类函数的正式名字，`EventListenerFn` 指向的就是它。

回调是更通用的叫法：你把自己的函数交给别人，别人在合适的时机替你调用。事件处理函数是一种回调，钩子函数也是一种回调。本书在讲具体类型时用“事件处理函数”，在讲“把函数交给别人”这个动作时用“回调”。

通知说的是事件的行为，不是函数的名字。宿主发布事件，总线把消息送到处理函数手里，这个过程叫通知。通知是单向的，发完就结束。

为什么返回 `void`？因为事件只负责通知，不负责回答。宿主说“我保存了”，它不关心索引插件有没有更新成功。如果它需要知道结果，那就不是通知，而是一次请求，应该用函数调用或者服务接口，不该用事件。

为什么有两个参数，而不是把类型和数据拼成一个字符串？因为一个事件处理函数可能订阅多个事件，它需要知道这次是哪一个。比如同一个函数既处理

`file.saved` 也处理 `file.deleted`，靠第一个参数区分。

为什么用 `const char*` 而不是 `std::string`？第 4 章讲过，跨动态库不能传 C++ 对象。`const char*` 的布局最简单，两个编译器看起来一样。`const` 表示处理函数只能读，不能改这块内存。

`type` 和 `payload` 指向的字符内容由宿主提供，只在这次调用期间有效。处理函数如果想把内容留到以后用，必须自己复制一份。

6.3 订阅：把函数地址交给总线

插件要收到事件，得先登记。登记的动作是“告诉总线：`file.saved` 发生时，调用我这个函数”。

总线内部按类型保存一组订阅者：

```
struct Listener {
    std::string owner;    // 哪个插件订阅的
    int priority;         // 数字小的先执行
    EventListenerFn fn;   // 事件处理函数的地址
};

std::map<std::string, std::vector<Listener>> listeners_;
```

`listeners_` 是一个键值表。键是事件类型，比如 `file.saved`；值是一个数组，里面放着所有订阅了这个类型的插件。

一条 `Listener` 有三个字段。`owner` 是插件名，卸载时按它清理；`priority` 决定执行顺序；`fn` 是事件处理函数的地址。

订阅函数 `on` 做三件事：

```
void EventBus::on(const std::string& type, const std::string& owner,
                  void (*fn)(const char*, const char*), int priority) {
    if (!fn) return;

    auto& list = listeners_[type];
    list.push_back({ owner, priority, fn });

    std::stable_sort(list.begin(), list.end(),
```

```
                    [](const Listener& a, const Listener& b) {
                        return a.priority < b.priority;
                    });
}
```

第一，检查 `fn` 是不是空。空指针不能调用，直接忽略。

第二，把这条订阅加到对应类型的数组末尾。`listeners_[type]` 如果这个类型还没有人订阅，会自动建一个空数组，然后 `push_back` 把新订阅放进去。

第三，按优先级排序。`std::stable_sort` 是稳定排序：优先级相同的订阅者，保持它们注册的先后顺序。如果用普通的 `sort`，相同优先级的顺序可能被打乱，今天 A 先、明天 B 先，行为就不确定了。

插件调用 `on` 时，`owner` 不需要自己填。宿主在插件初始化时知道当前是哪个插件，会替它填上。这样插件没法冒充别人订阅，卸载时也不会误删别人的订阅。

6.4 发布：总线挨个调用

保存完成时，宿主调用 `emit`：

```
void EventBus::emit(const std::string& type, const std::string& payload) {
    auto it = listeners_.find(type);
    if (it == listeners_.end()) return;

    auto snapshot = it->second;
    for (const auto& listener : snapshot) {
        listener.fn(type.c_str(), payload.c_str());
    }
}
```

第一步，按类型找到订阅者数组。找不到说明没人关心这个事件，直接返回，什么也不做。事件总线允许“发了但没人听”，这是正常情况。

第二步，把数组复制一份，叫 `snapshot`（快照）。

为什么要复制？因为事件处理函数执行时可能取消订阅。比如索引插件在处理 `file.saved` 时发现自己要退出，调用 `remove_owner` 把自己删了。如果直接遍历

原数组，删除元素会让后面的元素前移，遍历的下标就会跳过一些订阅者，或者访问到已经失效的内存。先复制一份，遍历的是稳定快照，删原数组不影响这一轮。

复制的代价是每个事件多一次数组拷贝。事件通常不频繁，这点开销可以接受。稳定性比省一次复制重要。

第三步，挨个调用。`type.c_str()` 和 `payload.c_str()` 把 `std::string` 转成 C 风格字符串，因为 `EventListenerFn` 接收的是 `const char*`。这两个指针只在调用期间有效，处理函数不能保存它们。

这里有一个隐藏的问题：如果某个处理函数抛异常，后面的订阅者就收不到事件，异常还会穿过 `emit`，继续往外传。第 6.9 节会讲怎么处理。

6.5 顺序：谁先谁后

顺序会直接影响结果，而且影响的方式不一定明显。

假设两个插件都监听 `file.saved`。A 负责把文件内容同步到云端，B 负责在本地生成缩略图。谁先谁后通常不影响结果，因为它们做的事互不干扰。

换一个组合。A 负责压缩文件，B 负责计算文件哈希。B 先算，算的是压缩前的哈希；A 先算，算的是压缩后的哈希。两个结果完全不同，而用户看到的可能只是“哈希偶尔对不上”。

宿主不能假设插件之间没有顺序依赖。必须提供优先级，并且把“数字小的先执行”写进文档。插件如果依赖某个顺序，应该显式声明优先级，而不是碰运气。

优先级数字怎么定？没有统一标准，但可以约定几个档位：`0` 到 `49` 留给系统内部，`50` 到 `99` 给需要抢先的插件，`100` 是默认值，`101` 以后表示“我希望晚一点”。这样大部分插件用默认值，少数有顺序需求的插件才有明确位置。

6.6 钩子：能改数据，也能阻止

事件只能通知，不能改变正在发生的事。宿主喊“我保存了”，这时候文件已经写完了，索引插件再想改内容也来不及。

但有些场景需要在事情发生之前插手。比如保存之前，插件想给内容加一行版权注释；打开文件之前，插件想检查用户有没有权限；绘制界面之前，插件想换一套配色。

这些场景需要**钩子**。钩子和事件很像，都是“到点了调用别人”，但钩子有两个额外的能力：可以改数据，可以用返回值阻止流程。

保存前钩子不是为了“正常编辑内容”。正常编辑发生在用户敲键盘的时候，走的是命令和编辑接口，改的是宿主内存里的文档，会进入撤销栈，界面上能立刻看到。

保存钩子处理的是另一件事：**准备写出去的那份内容**。同一份文档，内存里是一回事，写到磁盘上可能是另一回事。

常见的三类用途。

规范化和格式化。 把 Windows 风格的换行统一成 Unix 风格（前者每行结尾两个字符，后者一个）、去掉行尾多余空格、给 Markdown 补 front matter、把 JSON 的键排序。这些操作如果每次按键都做，会打断输入、污染撤销栈；放在保存时做一次最合适。

校验和阻止。 检查内容里有没有未填的占位符、有没有泄露密钥、文件是不是超过大小限制。发现问题就返回 `false`，保存中止，用户得到提示。这类钩子不需要修改内容，只需要说“不许存”。

附加信息和转换。 在文件头加“最后修改时间”，保存时顺便生成一份备份，或者把内容加密、压缩后再写。写出去的是密文，内存里仍然是明文。

如果没有这些需求，钩子链可以是空的，保存就是直接写文件。钩子是“可选的处理步骤”，不是保存的必需环节。

还要区分两个“改内容”：

- 编辑时的修改：改宿主内存里的文档，用户看得见，可以撤销。
- 保存时的修改：改准备写出去的副本，通常不回写文档，界面上看不到。

宿主在写文件之前会依次调用这些钩子。先看其中一个：它接收内容，在开头加一行注释，然后返回 `true` 表示继续：

```
std::vector<std::function<bool(std::string&)>> g_before_save;

g_before_save.push_back([](std::string& content) {
    content = "// 由宿主添加\n" + content;
    return true;
});
```

这里用 `std::function<bool(std::string&)>`，意思是“接收一个字符串引用、返回 bool 的函数”。`std::string&` 前面的 `&` 表示引用，函数拿到的是原字符串本身，不是副本，所以修改 `content` 会影响外面的内容。

为什么这里可以用 `std::string`？因为这段钩子只在宿主内部使用，不跨动态库。第 4 章那条“只传简单类型”的规则，针对的是跨动态库的边界。宿主自己的函数之间，用 `std::string`、`std::function` 都没问题。

返回 `true` 表示“我处理完了，继续”。如果某个钩子返回 `false`，宿主就停止保存：

```
for (auto& hook : g_before_save) {
    if (!hook(content)) {
        std::printf("保存被钩子阻止\n");
        return;
    }
}
```

这段循环就是钩子链：按顺序执行每个钩子，任何一个说“不”，整个保存就中止。

注意 `content` 是宿主在保存时临时复制的一份，不是文档对象本身。钩子改的是这份副本，改完直接写文件，不会回写文档。所以保存之后，编辑器里看不到多出来的那行注释，只有磁盘上的文件多了。

6.7 钩子链：顺序、短路、数据

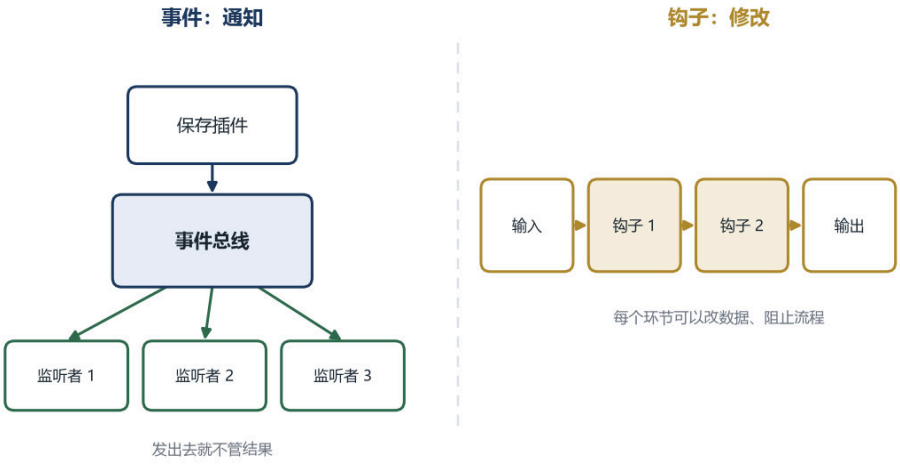


图 6-1　事件把消息广播给所有订阅者，钩子把数据串成一条管道

钩子链有三个要点。

顺序。 钩子按加入的顺序执行。第一个钩子加注释，第二个钩子去空行，结果和反过来不一样。加注释可能引入空行，去空行可能删掉注释里的换行。顺序要在文档里写清楚。

短路。 任何一个钩子返回 `false`，后面的钩子不再执行，保存中止。这叫短路。它适合“权限检查”“内容校验”这类场景：条件不满足，就没必要继续。

数据。 每个钩子接收的是同一个字符串的引用，前一个钩子改完，后一个钩子看到的就是改过的内容。这和管道型扩展点是同一个模型：前一个的输出是后一个的输入。

如果钩子要由插件提供，跨动态库传递，就不能用 `std::string&`，得换成第 4 章那种 C 风格的结构：一块 `char*` 缓冲区加一个长度，返回 `int` 表示成功失败。原理一样，只是数据的表示方式受边界限制。

6.8 三种钩子

按调用位置，钩子可以分三类。

前置钩子。 在操作执行前调用，可以修改输入、检查条件、阻止操作。保存前的注释、打开前的权限检查，都属于这一类。上面 `g_before_save` 就是前置钩子。

后置钩子。 在操作完成后调用，通常只做通知和清理，返回值被忽略。保存后的索引更新、上传完成后的通知，属于这一类。后置钩子不该抛异常，也不该阻止流程，因为事情已经做完了。

环绕钩子。 把原操作包在中间，可以在前后都做事，也可以决定要不要执行原操作。计时、重试、事务，属于这一类。

环绕钩子的写法稍微绕一点：

```
using Operation = std::function<bool()>;

bool run_with_timing(const Operation& core) {
    auto start = std::chrono::steady_clock::now();

    bool ok = core();   // 执行真正的操作

    auto end = std::chrono::steady_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end -
start).count();
    std::printf("保存耗时 %lld 毫秒\n", (long long)ms);
    return ok;
}
```

`Operation` 是“不接收参数、返回 bool 的函数”。`run_with_timing` 接收一个操作，先记时间，再调用它，最后打印耗时。原操作被夹在中间，所以叫环绕。

环绕钩子也可以选择不调用 `core()`，直接返回 `false`，这样原操作根本不会执行。权限检查、二次确认可以用这种方式。

6.9 异常不能穿过边界

C++ 有异常机制，`throw` 抛出，`try/catch` 捕获。但在插件系统里，异常穿过动态库边界非常危险。

原因在于异常的实现依赖运行库。Visual Studio 和 GCC 的异常结构不一样，甚至同一编译器不同版本的运行库也可能不兼容。插件里抛出的异常，宿主可能根本接不住，程序直接终止。

所以插件接口有一条硬规则：**异常不能穿过边界。** 插件里的函数要用返回值表示成功失败，内部该捕获的异常在内部捕获掉。

```
extern "C" PLUGIN_EXPORT int plugin_init(const PluginApi* api) {
    try {
        // 可能抛异常的逻辑
        return 0;
    } catch (const std::exception& e) {
        std::printf("init failed: %s\n", e.what());
        return 1;
    } catch (...) {
        std::printf("init failed: unknown error\n");
        return 2;
    }
}
```

`catch (...)` 捕获所有类型的异常，包括那些不是 `std::exception` 的。它是最后一道防线，保证异常不会漏出去。

宿主调用插件的函数时也一样。事件处理函数和钩子函数都可能来自插件，宿主应该用 `try/catch` 包住，避免一个插件出错影响其他订阅者：

```
for (const auto& listener : snapshot) {
    try {
        listener.fn(type.c_str(), payload.c_str());
    } catch (...) {
        std::printf("事件处理失败: %s\n", listener.owner.c_str());
    }
}
```

这段代码和 6.4 的 `emit` 相比，只多了一个 `try/catch`。但这几行很重要：一个订阅者写错了，不该让整条广播中断。

6.10 事件和钩子怎么配合

用户输入 `save` 后，宿主开始保存。一次完整的保存会同时用到钩子和事件。

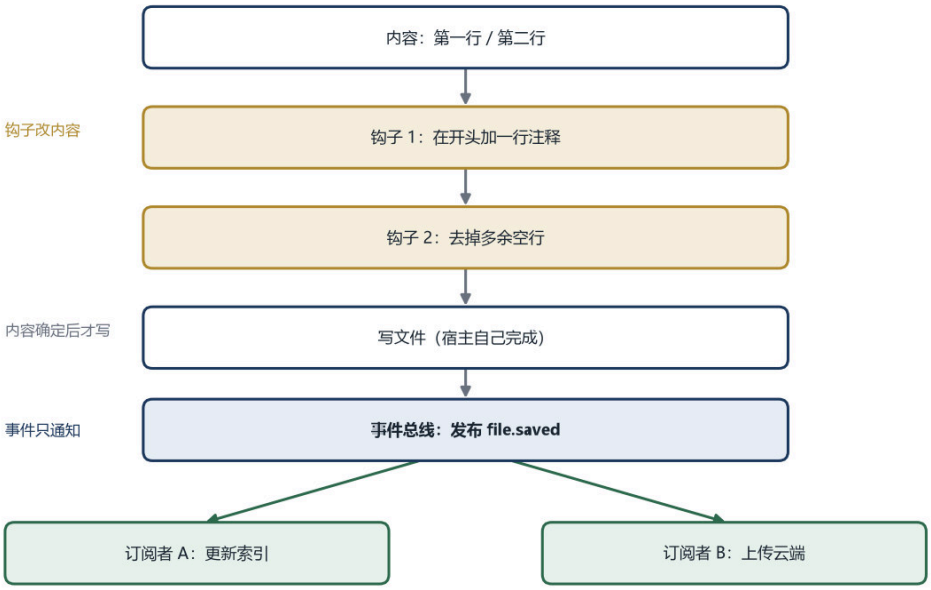


图 6-2　一次保存经过钩子和事件

顺序是这样的：

1. 宿主收到 `save` 命令，取出要保存的内容
2. 依次执行前置钩子，钩子可以修改内容，也可以阻止保存
3. 内容确定后，宿主写文件
4. 写完之后，发布 `file.saved` 事件
5. 索引插件、云插件收到事件，各自更新

钩子负责“改”，事件负责“通知”。分工明确，流程就能看懂。

如果反过来，用事件去修改内容，就会出问题：订阅者不知道自己的修改会不会被后面的人覆盖，也不知道事件是按什么顺序送的。事件的设计假设是“只读”，一旦有人往里写，整个模型就乱了。

判断标准很简单。插件只需要知道“发生了什么”，用事件；插件需要改变“正在发生的事”，用钩子。

6.11 一个容易忽略的问题：重入

假设保存钩子里，某个插件修改了文档内容。文档内容一变，又触发了一次“文档已修改”事件。宿主的自动保存定时器收到这个事件，又发起一次保存，保存又调用钩子。程序陷入循环。

这类问题叫**重入**：同一个流程还没结束，又被自己触发了一次。

最直接的解决办法是在流程里加一个标记：

```
static bool g_saving = false;

bool save_document(const std::string& path, std::string& content) {
    if (g_saving) {
        std::printf("已经在保存，忽略这次请求\n");
        return false;
    }

    g_saving = true;
    bool ok = do_save(path, content);
    g_saving = false;
    return ok;
}
```

`g_saving` 是一个开关。进入保存时打开，结束时关闭。保存过程中再有人请求保存，直接拒绝。

这个写法很简单，但有一个漏洞：如果 `do_save` 中途抛异常，`g_saving` 就永远不会被关掉，之后所有保存都会被拒绝。第 11 章会讲用 RAII 解决这个问题：把开关包在一个对象里，对象析构时自动关闭，不管从哪条路径退出。

6.12 还差什么

事件和钩子让插件能协作，但所有东西仍然在同一个进程里。插件里一个数组越界，宿主跟着崩溃。插件里一个死循环，界面卡住，用户连禁用按钮都点不到。

下一章处理这个问题。先把“进程”讲清楚，再把插件放进独立进程，最后讨论通信、超时和权限。

第 7 章 插件出错怎么办

本章的完整代码和编译说明在 `codes/ch07/`。

7.1 C++ 的崩溃为什么特别致命

前面几章的插件和宿主在同一个程序里。这意味着插件犯的错，宿主一起承担。

C++ 里有几类错误，后果特别严重。

数组越界。 你有一个长度为 10 的数组，却写了第 15 个位置。那 5 个位置可能属于别的变量。写进去之后，别的变量的值被悄悄改掉，程序在几分钟后以一个完全无关的地方崩溃。排查时看到的现场，和真正的错误地点隔着十万八千里。

空指针。 一个指针没有指向任何有效对象，你却通过它去读数据。轻则读到垃圾，重则直接触发操作系统的保护机制，程序立刻终止。

释放后再使用。 一块内存已经还给了系统，你手里还留着一个指向它的指针，又用它去读写。这块内存可能已经被分配给别的对象，你改的是别人的数据。

栈溢出。 函数递归太深，或者在一个函数里开了巨大的局部数组，把栈空间用完。程序同样立刻终止。

这些错误有一个共同点：它们不是“返回一个错误码”，而是直接让程序消失。C++ 的 `try/catch` 接不住它们，因为它们在语言层面之下，属于操作系统检测到的非法内存访问。

7.2 程序运行起来之后

要理解怎么隔离，先要理解程序运行起来之后是什么样子。

你写的 `.cpp` 文件躺在硬盘上，它是一个文件。双击它，操作系统会做几件事：在内存里划出一块地方，把程序的代码和数据装进去，创建一个执行环境，然后从 `main` 开始执行。这个“装进内存、正在运行的程序”，叫**进程**。

进程有自己的内存。这块内存里放着它的代码、全局变量、堆、栈。操作系统给每个进程分配独立的地址空间，一个进程不能直接读写另一个进程的内存。你去读，操作系统会拦住你，然后终止你的程序。

这就像一栋楼里的房间。每个进程是一间房，房里的东西别人进不来。你可以随便折腾自己房间，把墙砸了也行，但砸不到隔壁。

进程之间要交换东西，不能直接伸手，只能通过门缝递纸条。这个“递纸条”的通道，叫**管道**或者**套接字**，统称进程间通信。

7.3 同一个进程里的保护为什么不够

有人会问，既然有 `try/catch`，为什么不能在插件外面包一层，出错就捕获？

因为 `try/catch` 只能捕获“主动抛出”的异常。数组越界、空指针解引用这类错误，不会抛出异常，它们直接触发操作系统的内存保护。程序收到一个信号，默认行为是立刻终止，`catch` 根本没机会执行。

死循环也接不住。一个 `while (true)` 不带退出条件，会一直占着处理器。宿主和插件在同一个进程，插件的循环不结束，宿主的主循环也得不到执行，界面就卡死了。

要兜住这两类问题，只有一个办法：把插件放到另一个进程里。插件崩溃，只死它自己；插件死循环，宿主可以直接把它杀掉。

7.4 把插件做成一个独立程序

做法很直接。插件不再编译成动态库，而是编译成一个独立的可执行文件，比如 `plugin_host.exe`。这个程序启动后，等待宿主发消息，收到消息就执行，执行完把结果写回去。

宿主这边，启动这个程序，拿到它的输入输出通道，像对话一样一问一答。

结构变成这样：

```
宿主进程
   ├── 加载插件清单
   ├── 启动 plugin_host.exe（每个插件一个）
   ├── 通过管道发消息
   └── 收结果、处理超时、必要时杀掉
插件进程
   ├── 读一行消息
   ├── 执行对应命令
   └── 写一行结果
```

插件进程里仍然可以加载动态库，也可以直接写死命令。关键是它和宿主之间隔着一道进程边界，崩溃不会传播。

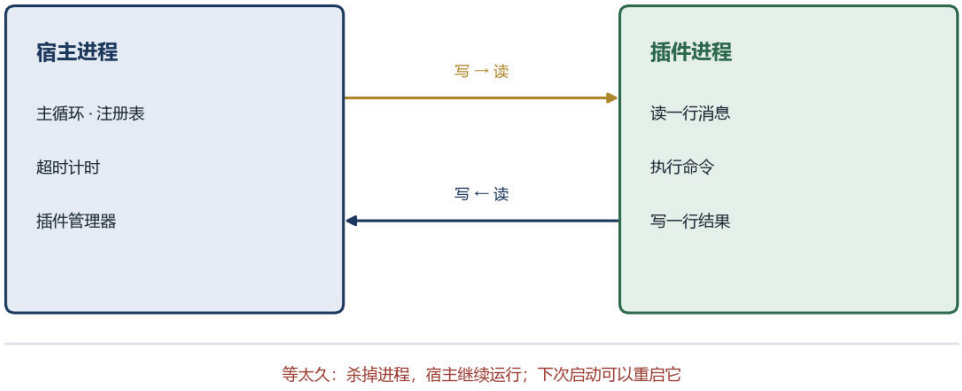


图 7-1　宿主和插件进程通过两根管道一问一答

7.5 两个程序怎么说话

进程之间传数据，最朴素的方式是管道。

管道是操作系统提供的一根“管子”，一头写，另一头读。写进去的是字节流，读出来的也是字节流。它不关心你写的是文字还是二进制，只负责搬运。

为了让两边都能理解，需要约定一种格式。最简单的是“一行一条消息”，每条消息是一段文本，用换行符分隔。

```
宿主 -> 插件:   run hello world
插件 -> 宿主:   ok 你好，世界
插件 -> 宿主:   error 命令不存在
```

每行第一个词表示类型，后面是内容。解析时按第一个空格切开，和命令行参数的切法一样。

把内存里的数据变成可以传输的字节，这个过程叫**序列化**。反过来叫**反序列化**。文本协议是最简单的序列化，缺点是体积大、解析慢；二进制协议更紧凑，但调试时看不懂。插件系统的消息通常不多，文本协议够用。

7.6 超时与强制结束

插件进程可能卡住。宿主不能一直等。

每次发完消息，宿主启动一个计时器。比如等三秒，三秒还没收到回复，就认为插件挂了。

处理方式有两种。温和一点的是发一条“取消”消息，给插件一个自己退出的机会。强硬一点的是直接杀掉进程。

Windows 上杀进程用 `TerminateProcess`，Linux 上用 `kill` 发送 `SIGKILL` 信号。杀掉之后，进程占用的内存、打开的文件都会被系统回收，不会泄漏。

杀掉之后要不要重启？可以重启，但不能无限重启。第一次崩溃后立即重启，第二次等一秒，第三次等五秒，之后逐渐拉长。短时间内崩溃太多次，就标记为“反复崩溃”，提示用户禁用。

7.7 权限：只给该给的

隔离解决了“崩溃不扩散”，但没有解决“插件能做什么”。一个独立进程仍然可以读写用户的文件、访问网络。

权限控制的思路是“默认什么都不给，需要什么单独授予”。

具体到进程，可以限制它能访问的目录、能连接的网络地址、能使用的内存大小。Windows 上可以用作业对象（Job Object）限制内存和进程数；Linux 上可以用 `setrlimit` 限制资源，用命名空间隔离文件系统。

这些机制对初学者来说比较深，这一章只需要知道原则：**插件进程应该以最小的权限运行，用到什么开什么，不用就关掉。**

一个具体的做法是，插件不直接接触文件路径，而是把请求发给宿主，宿主检查路径是否在允许的目录里，再代为读写。插件拿不到真实路径，也就没法越界。

7.8 一个最小的进程插件

下面这段代码只做一件事：启动一个子程序，发一句 `ping`，读一句回复。它不完整，但包含了进程隔离的全部关键步骤。

先看子程序 `plugin_host.cpp`：

```
#include <cstdio>
#include <iostream>
#include <string>

int main() {
    std::string line;
    while (std::getline(std::cin, line)) {
        if (line == "ping") {
            std::printf("pong\n");
            std::fflush(stdout);
        } else if (line == "quit") {
            break;
        } else {
            std::printf("error unknown\n");
            std::fflush(stdout);
        }
    }
    return 0;
}
```

它从标准输入读一行，判断内容，往标准输出写一行。没有网络，没有复杂协议，就是对话。

宿主这边，Windows 的写法：

```
#include <windows.h>
#include <cstdio>
#include <cstring>

int main() {
    SECURITY_ATTRIBUTES sa{};
    sa.nLength = sizeof(sa);
    sa.bInheritHandle = TRUE;

    HANDLE in_read = nullptr, in_write = nullptr;
    HANDLE out_read = nullptr, out_write = nullptr;
    CreatePipe(&in_read, &in_write, &sa, 0);
    CreatePipe(&out_read, &out_write, &sa, 0);

    STARTUPINFOA si{};
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput = in_read;
    si.hStdOutput = out_write;
    si.hStdError = out_write;

    PROCESS_INFORMATION pi{};
    char cmd[] = "plugin_host.exe";
    if (!CreateProcessA(nullptr, cmd, nullptr, nullptr, TRUE, 0,
                        nullptr, nullptr, &si, &pi)) {
        std::printf("启动失败: %lu\n", GetLastError());
        return 1;
    }

    CloseHandle(in_read);
    CloseHandle(out_write);

    const char* msg = "ping\n";
    DWORD written = 0;
    WriteFile(in_write, msg, (DWORD)std::strlen(msg), &written, nullptr);

    char buffer[256]{};
    DWORD got = 0;
    ReadFile(out_read, buffer, sizeof(buffer) - 1, &got, nullptr);
    std::printf("子进程回复: %s", buffer);

    CloseHandle(in_write);
    WaitForSingleObject(pi.hProcess, 3000);
    TerminateProcess(pi.hProcess, 0);

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    CloseHandle(out_read);
    return 0;
}
```

这段代码有点长，但每一步都对应一个具体目的。

`SECURITY_ATTRIBUTES` 里的 `bInheritHandle = TRUE`，表示“这个管道的把手可以被子进程继承”。不设这个，子进程拿不到管道。

`CreatePipe` 创建两根管道。第一根给宿主写、子进程读，第二根给子进程写、宿主读。每根管道有两个把手，一头读一头写。

`STARTUPINFOA` 告诉系统：子进程的标准输入输出不要用控制台，用我指定的这两个把手。`STARTF_USESTDHANDLES` 是“使用自定义把手”的开关。

`CreateProcessA` 启动子进程。第五个参数 `TRUE` 表示“允许继承把手”。

启动之后，宿主把自己不需要的那两头关掉。`in_read` 是子进程读的那头，宿主不用；`out_write` 是子进程写的那头，宿主也不用。不关掉的话，管道永远有一端开着，读数据时会一直等，等不到结束。

`WriteFile` 写一句 `ping\n`。`ReadFile` 读回复。注意 `ReadFile` 是阻塞的，子进程不写，它就一直等。真实系统要用 `PeekNamedPipe` 轮询或者异步 IO 来配合超时。

最后 `WaitForSingleObject` 等子进程退出，最多等三秒。没退出就 `TerminateProcess` 强制结束。

7.9 Linux 上对应的写法

Linux 的思路一样，函数名不同。

步骤	Windows	Linux
创建管道	`CreatePipe`	`pipe`
启动进程	`CreateProcess`	`fork` + `execv`
重定向输入输出	`STARTUPINFO`	`dup2`
读	`ReadFile`	`read`
写	`WriteFile`	`write`
等待	`WaitForSingleObject`	`waitpid`
强制结束	`TerminateProcess`	`kill` + `SIGKILL`

`fork` 会复制当前进程，`execv` 把复制出来的进程替换成另一个程序。`dup2` 把管道的两端接到标准输入输出上。这三步合起来，效果和 Windows 的 `CreateProcess` 加 `STARTUPINFO` 一样。

书里不展开 Linux 的完整代码，因为逻辑和 Windows 一一对应，读者理解了一个，另一个可以查文档写出来。

7.10 什么时候不该隔离

进程隔离不是免费的。启动一个进程要几十毫秒，每次通信要序列化，调试要同时看两个进程的日志。

下面这些情况，放在同一个进程里更合适。

插件完全由自己开发，代码经过审核，不会执行用户输入的任意代码。

操作频率极高，比如每一帧都要调用的绘制指令。跨进程的消息开销会直接吃掉性能预算。

插件需要共享大量内存中的对象，复制到另一个进程既慢又容易不一致。

插件生命周期很短，比如一次性的格式转换，启动进程的时间比干活还长。

一个折中的做法是分级。可信插件放在宿主进程，第三方插件各自独立，高风险操作再单独隔离。这样既能保证安全，又不会让所有插件都背上通信开销。

7.11 这一章解决了什么

到这儿，第 1 章提出的三个问题都有了着落。

加功能不用改主程序，插件是独立的动态库或者独立程序。代码不在一个文件里，插件可以单独编译、单独发布。插件崩溃不会带走宿主，因为它在另一个进程里。

代价是复杂度上来了。要写加载器、注册表、事件总线、进程管理、通信协议、超时处理。这也是为什么大型软件的架构看起来那么复杂，它不是故意绕弯，而是每个零件都在解决一个具体问题。

接下来的第三篇，把这些零件变成可复制的工程。先解决一个很实际的问题：每写一个新插件，都要手动建目录、写清单、配构建，太麻烦。下一章用一条命令生成。

第三篇 把代码变成工程

能跑起来只是开始。一个插件系统要被人持续使用，还需要模板、测试、构建、打包、更新。这一篇处理这些“代码之外”的事。

第 8 章用模板和一条命令，把新插件的创建过程固定下来。

第 9 章让插件脱离宿主独立开发、独立测试、自动构建。

第 10 章把插件打成包，签名、安装、更新、回滚。

第 8 章 一条命令生成插件

本章的完整代码和编译说明在 `codes/ch08/`。

8.1 每写一个插件都要重复一遍

做到现在，写一个新插件要手动做这些事：建目录、写 `plugin_api.h` 的引用、写三个导出函数、写命令数组、配编译命令、建 `plugins` 子目录。

这些事每次都一样，只有名字和命令内容不同。重复做十遍，迟早会漏一步。漏了 `extern "C"`，插件加载时找不到符号；忘了导出宏，动态库是空的；编译参数少一个，插件和宿主对不上。

解决办法是把这套结构做成模板，用一条命令生成。生成出来的东西能直接编译、能直接加载，作者只需要往里填业务。

8.2 模板里放什么

一个插件模板至少包含六样东西。

接口头文件。 `plugin_api.h` 的副本，或者一个指向公共头文件目录的引用。插件作者需要它才能编译。

插件源码。 一个空的 `plugin.cpp`，已经写好三个导出函数和命令数组的骨架。

构建配置。 CMake 文件，说明这是一个动态库、用 C++17、输出到哪里。

清单文件。 一个简单的文本或者 JSON，记录插件名字、版本、作者、依赖。第 10 章打包时会用到。

说明文档。 一个 `README.md`，写清楚这个插件做什么、怎么编译、怎么安装。

一个最小示例。 一个能跑的 `hello` 命令，作者可以照着改成自己的。

这六样东西不需要很复杂。模板的价值不在代码多，而在于“结构正确”。作者拿到就能编译，不用先研究怎么配置。

8.3 目录结构

```
my-plugin/
  CMakeLists.txt
  plugin.json
  README.md
  include/
    plugin_api.h
  src/
    plugin.cpp
```

`include` 放公共头文件，`src` 放源码。`CMakeLists.txt` 是构建说明，`plugin.json` 是清单，`README.md` 是给人看的。

这个结构刻意保持扁平。层级越深，新作者越容易放错位置。等插件真的变大，再按功能拆子目录。

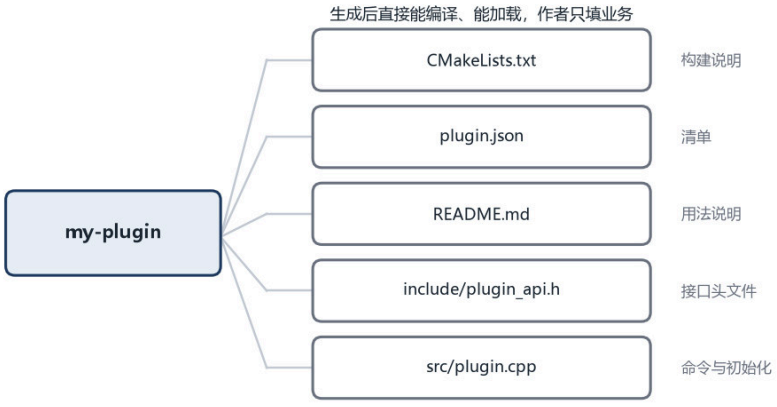


图 8-1　生成出来的插件目录

8.4 CMake 是什么

C++ 的编译命令可以很长，尤其是文件多了、要链接不同的库、要区分 Windows 和 Linux 的时候。每次都手写命令，既容易错，也没法跨平台。

CMake 是一个“生成构建文件”的工具。你写一份 `CMakeLists.txt`，描述“这个项目有哪些源文件、要生成什么、依赖什么”，CMake 根据当前平台生成对应的构建文件。Windows 上生成 Visual Studio 的工程，Linux 上生成 Makefile。

一个最小的插件构建文件：

```
cmake_minimum_required(VERSION 3.16)
project(my_plugin LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_library(my_plugin SHARED src/plugin.cpp)
target_include_directories(my_plugin PRIVATE include)
```

`cmake_minimum_required` 声明需要的最低 CMake 版本。`project` 给项目起名字，`LANGUAGES CXX` 表示这是一个 C++ 项目。

`set(CMAKE_CXX_STANDARD 17)` 要求用 C++17 标准。`CMAKE_CXX_STANDARD_REQUIRED ON` 表示“如果编译器不支持，就直接报错”，而不是悄悄退回旧标准。

`add_library(... SHARED ...)` 表示生成动态库。`SHARED` 是关键，不写的话默认生成静态库。`target_include_directories` 把头文件目录加进搜索路径，这样源码里可以写 `#include "plugin_api.h"`。

使用的时候，在项目目录里执行：

```
cmake -B build
cmake --build build
```

第一行在 `build` 目录里生成构建文件，第二行真正编译。生成的动态库在 `build` 目录下，Windows 是 `my_plugin.dll`，Linux 是 `libmy_plugin.so`。

8.5 用脚本生成

生成器可以用任何语言写。这里用 Python，因为它做文件操作最短，而且 Windows 和 Linux 都自带或者容易安装。

先看完整脚本，再逐段解释：

```
import os
import shutil
import sys

TEXT_EXT = (".cpp", ".h", ".txt", ".json", ".md")

def render(text, values):
    for key, value in values.items():
        text = text.replace("{{" + key + "}}", value)
    return text

def main():
    if len(sys.argv) < 2:
        print("用法: python new_plugin.py <name>")
        return

    name = sys.argv[1]
    target = os.path.join("plugins", name)

    if os.path.exists(target):
        print("目录已存在:", target)
        return

    shutil.copytree("templates/plugin", target)

    for root, dirs, files in os.walk(target):
        for filename in files:
            if not filename.endswith(TEXT_EXT):
                continue
            path = os.path.join(root, filename)
            with open(path, "r", encoding="utf-8") as f:
                text = f.read()
            with open(path, "w", encoding="utf-8") as f:
                f.write(render(text, {"name": name}))

    print("已创建", target)
    print("下一步: cd", target, "&& cmake -B build && cmake --build build")

if __name__ == "__main__":
    main()
```

`sys.argv` 是命令行参数列表。第一个是脚本名，第二个开始是用户输入。`len(sys.argv) < 2` 表示用户没给名字，打印用法后退出。

`os.path.join` 把目录和名字拼成路径。它会根据系统自动选斜杠，Windows 用反斜杠，Linux 用正斜杠。

`os.path.exists` 检查目录在不在。存在就报错，不覆盖。这个检查很重要，生成器最怕的就是手滑覆盖别人的代码。

`shutil.copytree` 复制整个模板目录。它会递归复制所有子目录和文件。

`os.walk` 遍历目录树，每次返回三个东西：当前目录、当前目录下的子目录列表、当前目录下的文件列表。

`filename.endswith(TEXT_EXT)` 检查文件扩展名是不是文本文件。图片、字体这类文件不能做文本替换，直接跳过。

`with open(...) as f` 是 Python 的文件写法，离开这个块时自动关闭文件，不用手动 `close`。

`render` 把文件里的 `{{name}}` 替换成实际名字。模板里的 `plugin.cpp` 可能写着 `return "{{name}}";`，生成后变成 `return "hello";`。

8.6 模板长什么样

`templates/plugin/src/plugin.cpp`：

```
#include "plugin_api.h"
#include <cstdio>

static void run_{{name}}(const char* args) {
    std::printf("{{name}} 命令执行，参数：%s\n", args ? args : "");
    std::fflush(stdout);
}

static const PluginCommand kCommands[] = {
    { "{{name}}", "{{name}} 插件的示例命令", run_{{name}} },
};

extern "C" PLUGIN_EXPORT const char* plugin_name() {
    return "{{name}}";
}

extern "C" PLUGIN_EXPORT int plugin_init(const PluginApi* api) {
    if (!api || api->version != PLUGIN_API_VERSION) return 1;
    for (const auto& cmd : kCommands) {
        api->register_command(&cmd);
```

```
    }
    return 0;
}

extern "C" PLUGIN_EXPORT void plugin_shutdown() {
}
```

生成 `hello` 插件后，`{{name}}` 全部变成 `hello`，函数名变成 `run_hello`，命令名变成 `hello`。

注意函数名里用了占位符。如果两个插件都叫 `run_command`，虽然它们在各自的动态库里，不会冲突，但调试时容易看混。带上插件名更清楚。

8.7 生成之后的检查

生成命令结束时，最好自动做几件事。

检查名字。 只允许小写字母、数字和短横线，不能为空，不能和已有目录重名。

编译一次。 调用 CMake 构建，确认模板生成的代码能通过编译。这一步能发现占位符替换错误、头文件路径错误。

检查产物。 确认动态库真的生成了，而且里面有那三个导出符号。

打印下一步。 告诉用户怎么编译、怎么安装、怎么运行。命令结束时，用户应该知道接下来做什么。

这四项里，编译检查最重要。很多模板看起来完整，一编译就报错，作者还没开始写业务就先花半小时修模板。

8.8 模板也会过时

模板不是生成一次就结束。接口升级后，模板里的代码可能还在用旧写法。作者用旧模板生成插件，一开始就踩坑。

解决办法是给模板编版本，并在生成的插件清单里记下来。

```
{
  "name": "hello",
  "version": "1.0.0",
  "template": "1.2.0",
  "api": 1
}
```

生成器可以检查这个字段，发现模板版本落后时提示升级。更进一步的工具可以提供 `upgrade` 命令，把旧结构迁移到新结构。

迁移通常分三步：比较版本、列出要改的文件、执行替换，最后编译测试。自动迁移不可能覆盖所有情况，遇到无法处理的改动时，应该生成一份说明，让作者手动完成。

8.9 模板不该做的事

模板很容易越做越大，最后变成一个框架。有几件事要克制。

不要预置业务逻辑。模板生成的是空壳加一个最小示例，不是半个产品。作者需要的是结构，不是一堆要删掉的代码。

不要绑定某一个构建工具。如果模板默认用 CMake，用别的工具的团队就要改。可以提供多个模板变体，但目录结构和清单必须一致。

不要在模板里写死版本号。接口版本、C++ 标准、依赖版本，都应该从一处配置读取，改的时候只改一个地方。

不要生成没有用的文件。每个文件都应该有明确用途，作者知道为什么需要它。为了“完整”塞进去的空目录、空配置，最后只会变成噪音。

下一章讨论插件怎样脱离主程序独立开发。模板给了它一个正确的开始，独立开发决定了它能不能持续演进。

第 9 章 让插件独立开发

本章的完整代码和编译说明在 `codes/ch09/`。

9.1 独立开发是什么意思

“插件可以独立开发”不只是“代码放在另一个文件夹”。真正的独立，是指作者不需要主程序的全部源码，也不需要每次启动整个主程序，就能写完、跑通、验证一个插件。

要做到这一点，三样东西必须从主程序里拿出来：接口头文件、一个能加载插件的假宿主、一套能自动跑的测试。

接口头文件如果只在主程序源码里，作者得把整个主程序下载下来。假宿主如果没有，作者每次调试都得启动完整软件，慢且不稳定。测试如果不能自动跑，每次改完只能靠手点，迟早漏掉。

9.2 把接口头文件单独放

现在 `plugin_api.h` 在宿主项目里。插件要引用它，得把路径写死，或者复制一份。复制会让两个版本漂移，改了宿主忘了改插件，编译出来的东西对不上。

正确的做法是把头文件做成一个独立的小包，目录结构大概是这样：

```
sdk/
  include/
    plugin_api.h
  README.md
  VERSION
```

宿主和插件都引用这个目录。宿主编译时加 `-I../sdk/include`，插件也加同样的路径。这样只有一份头文件，改一处两边都生效。

`sdk` 里只放“双方都同意的东西”：接口声明、结构体定义、常量、版本号。不放实现，不放只有宿主才需要的内部类型。这个目录就是第 4 章说的“合同”。

真实项目里，`sdk` 可以单独打成一个压缩包，带版本号发布。插件作者下载 `sdk-1.2.0.zip`，解压后就能编译。宿主升级接口时，发布新的 `sdk`，插件按需升级。

9.3 假宿主

假宿主是一个很小的程序，它实现 `PluginApi`，加载指定插件，把插件注册的命令打印出来。它不启动界面，不读配置，不连接网络，只验证“插件能不能被加载、能不能注册”。

```
#include "plugin_api.h"
#include <cstdio>
#include <string>

#ifdef _WIN32
#include <windows.h>
#define LIB_OPEN(p) LoadLibraryA(p)
#define LIB_SYM(h, n) ((void*)GetProcAddress(h, n))
#else
#include <dlfcn.h>
#define LIB_OPEN(p) dlopen(p, RTLD_NOW)
#define LIB_SYM(h, n) dlsym(h, n)
#endif

static void on_command(const PluginCommand* cmd) {
    std::printf("  命令: %-10s %s\n", cmd->name, cmd->help ? cmd->help : "");
}

static void on_log(const char* message) {
    std::printf("  日志: %s\n", message ? message : "");
}

int main(int argc, char** argv) {
    if (argc < 2) {
        std::printf("用法: mock_host <plugin.dll>\n");
        return 1;
    }

    auto handle = LIB_OPEN(argv[1]);
    if (!handle) {
        std::printf("加载失败: %s\n", argv[1]);
        return 2;
    }
```

```
    auto init = (int(*)(const PluginApi*))LIB_SYM(handle, "plugin_init");
    if (!init) {
        std::printf("缺少 plugin_init\n");
        return 3;
    }

    PluginApi api{};
    api.size = sizeof(PluginApi);
    api.version = PLUGIN_API_VERSION;
    api.register_command = on_command;
    api.log_info = on_log;

    std::printf("加载 %s\n", argv[1]);
    int rc = init(&api);
    std::printf("init 返回 %d\n", rc);
    return rc;
}
```

这个程序不到六十行，但它给了插件作者一个完整的验证环境。编译出插件后，直接 `mock_host my_plugin.dll`，就能看到注册了哪些命令、初始化有没有报错。

假宿主不追求和真宿主一模一样。它只覆盖接口承诺的行为。真宿主的内部实现、界面细节、配置读取，都不属于合同，也不应该在插件的单元测试里依赖。

9.4 自己写一个测试框架

C++ 没有自带测试框架，但写一个最小的不难。

```
#include <cstdio>

static int g_failed = 0;

#define CHECK(condition)                                                    \
    do {                                                                    \
        if (!(condition)) {                                                 \
            std::printf("FAIL %s:%d  %s\n", __FILE__, __LINE__, #condition); \
            ++g_failed;                                                     \
        }                                                                   \
    } while (0)

static int add(int a, int b) { return a + b; }

int main() {
    CHECK(add(2, 3) == 5);
    CHECK(add(-1, 1) == 0);
```

```
    CHECK(add(0, 0) == 0);
    std::printf("%s\n", g_failed == 0 ? "全部通过" : "有失败");
    return g_failed == 0 ? 0 : 1;
}
```

`#define CHECK(...)` 定义一个宏。宏在编译前展开，把一段代码原样插到调用的地方。

`do { ... } while (0)` 是一个常用技巧。它把多条语句包成一个整体，展开后可以安全地放在 `if` 后面，不会因为分号出问题。

`__FILE__` 和 `__LINE__` 是编译器提供的宏，分别展开成当前文件名和行号。测试失败时，能直接定位到哪一行。

`#condition` 里的 `#` 是“字符串化”：把宏参数原样变成字符串。`CHECK(a == b)` 失败时，打印出来的就是 `a == b`，不用再手写说明。

`g_failed` 记录失败次数。主函数最后根据它返回 0 或 1，自动检查通过退出码判断成败。

这个框架只有十几行，够用了。等测试多起来，可以换成 Catch2 或者 GoogleTest，但那是以后的事。

9.5 本地联调

插件开发过程中，有三种跑法，耗时依次增加。

只跑测试。 改完代码，编译测试程序，运行。几百毫秒出结果。适合验证纯逻辑，比如命令解析、字符串处理、数据计算。

用假宿主跑。 编译出动态库，用 `mock_host` 加载，看注册和初始化。几秒出结果。适合验证接口对接，比如命令有没有注册上、日志有没有输出。

装进真宿主。 把动态库复制到主程序的 `plugins` 目录，启动主程序，手动点一遍。分钟级。适合最终验收。

三种跑法都要有，不能只有最后一种。只有最后一种，开发一个插件要反复启动完整软件，效率很低。

本地联调时，动态库经常被占用。Windows 上，一个加载中的 DLL 不能覆盖。改完代码重新编译，可能报“无法写入文件”。解决办法是先关闭宿主，或者编译到另一个文件名，再替换。

Linux 上通常可以直接覆盖正在使用的 `.so`，因为文件被替换后，已经打开的进程仍然指向旧文件，新进程用新文件。这个差异在跨平台开发时会遇到。

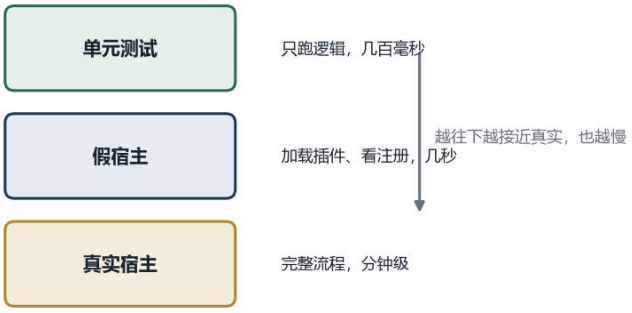


图 9-1　插件开发的三个循环，越往下越接近真实

9.6 持续集成

插件作者可以在每次提交后自动跑一遍检查。GitHub 提供免费的自动构建服务，配置文件放在 `.github/workflows/ci.yml`。

```yaml
name: build

on: [push, pull_request]

jobs:
  linux:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B build -DCMAKE_BUILD_TYPE=Release
      - run: cmake --build build
      - run: ctest --test-dir build --output-on-failure

  windows:
    runs-on: windows-latest
```

```
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B build
      - run: cmake --build build --config Release
```

`yaml` 是一种用缩进表示层级的配置格式。这段配置的意思是：每次有人提交代码，就在 Linux 和 Windows 各启动一台机器，检出代码、配置、编译、跑测试。

`on: [push, pull_request]` 表示触发条件：推送代码或者发起合并请求。

`runs-on` 指定操作系统。两个任务并行跑，任何一个失败都会在页面上标红。

`cmake -B build` 在 `build` 目录生成构建文件。`cmake --build build` 执行编译。`ctest` 跑测试。

CI 的价值在于“在别人的机器上也能编译”。你本地能过，不代表别人能过。不同编译器、不同标准库版本、不同路径分隔符，都会暴露问题。

9.7 版本与依赖

插件和宿主之间的版本关系，要在清单里写清楚。

```
{
  "name": "hello",
  "version": "1.2.0",
  "api": 1,
  "requires": {
    "host": ">=1.0.0 <2.0.0"
  }
}
```

`api` 是接口版本，对应 `PLUGIN_API_VERSION`。`requires.host` 是宿主版本范围。

宿主加载时检查这两项。`api` 不等于当前版本，直接拒绝；宿主版本不在范围内，也拒绝，并提示用户升级。

插件之间也可能有依赖。比如一个“云同步”插件依赖“存储”插件提供的服务。清单里可以写：

```
{
  "requires": {
    "host": ">=1.0.0 <2.0.0",
    "plugins": ["storage"]
  }
}
```

宿主要在加载前检查这些插件在不在，不在就跳过，并给出提示。加载顺序要按依赖排，第 10 章会讲。

版本号用三个数字，主版本加一表示不兼容，次版本加一表示增加功能但兼容，修订号加一表示修 bug。这个约定叫语义化版本，插件和宿主都遵守，用户才能一眼看出升级会不会出问题。

9.8 插件作者的日常

把上面的东西合起来，一个插件作者的日常大概是这样。

拉取 `sdk`，用生成器创建插件骨架。改 `plugin.cpp`，加自己的命令。跑 `cmake --build`，编译。跑 `mock_host`，看注册结果。写几个 `CHECK`，验证逻辑。提交，CI 自动跑 Linux 和 Windows。打标签，发布新版本。

整个过程不需要主程序的源码，也不需要主程序团队配合。这就是“独立开发”的实际含义。

下一章讨论插件做完之后怎么到达用户手里。打包、签名、安装、更新，这几步决定了插件生态能不能持续运转。

第 10 章 打包、分发与更新

本章的完整代码和编译说明在 `codes/ch10/`。

10.1 一个插件包长什么样

开发目录里有源码、构建文件、测试，这些不能直接给用户。用户需要的是一个确定的、能检查的、放到指定位置就能用的东西。

一个插件包通常是一个压缩文件，里面包含三部分：

```
hello-1.2.0.zip
  plugin.json        清单
  hello.dll          动态库（Linux 上是 hello.so）
  README.md          说明
```

清单放在最外层，宿主解压后先找它。动态库的名字不要求和插件名一样，清单里写清楚就行。说明文档给人看，不影响加载。

打包时要保证“同样的源码，每次打出来的包一样”。这叫确定性。影响确定性的东西包括文件顺序、时间戳、压缩参数。打包脚本应该按文件名排序，固定时间戳，不要每次都写入当前时间。这样两次构建的哈希才一致，缓存和差分更新才有意义。

10.2 哈希：给文件算一个指纹

哈希是一个函数，输入任意长度的数据，输出一段固定长度的数字。同样的输入永远得到同样的输出，输入改一个字节，输出就完全不同。

哈希的用途是校验。你下载一个文件，算一遍哈希，和作者公布的值比。一样，说明文件完整；不一样，说明路上被改了或者传坏了。

下面是一个简单的哈希实现，叫 FNV-1a：

```
#include <cstdint>
#include <string>

std::uint64_t fnv1a(const std::string& data) {
    std::uint64_t hash = 14695981039346656037ULL;
    for (unsigned char byte : data) {
        hash ^= byte;
        hash *= 1099511628211ULL;
    }
    return hash;
}
```

`std::uint64_t` 是 64 位无符号整数，能表示很大的数。`ULL` 后缀表示这个字面量是无符号长整数。

`hash ^= byte` 是异或运算。异或的规则是“相同为 0，不同为 1”，它把当前字节混进哈希值里。`hash *= 1099511628211ULL` 是乘以一个质数，让结果散开。这两步重复，每个字节都会影响最终结果。

这个哈希不算安全。它容易被故意构造出碰撞，也就是两份不同的数据算出同一个值。它适合校验“文件有没有传坏”，不适合防篡改。防篡改要用 SHA-256 这类密码学哈希，标准库没有，需要引入 OpenSSL 之类的库。

书里用 FNV 是为了让读者看清哈希的基本形状。真实项目里，校验完整性用 SHA-256，签名用 Ed25519 或者 RSA。

10.3 签名：确认是谁做的

哈希只能说明“内容没变”，不能说明“是谁做的”。攻击者可以改内容，再改哈希，你比对不出来。

签名解决这个问题。作者有一对钥匙，私钥自己保存，公钥公开。作者用私钥对哈希做运算，得到一个签名，附在包里。用户用公钥验证签名，验证通过就说明“这个包确实由持有私钥的人签过，且内容没变”。

私钥和公钥是一对数学上配对的数。用私钥算出来的东西，只有对应的公钥能验通过；别人没有私钥，造不出能通过的签名。

生活里的类比是印章。你有一枚只有自己有的章，盖在文件上，别人可以对照你的印鉴确认真假。别人仿造不了你的章，也就伪造不了你的签名。

具体到插件包，流程是这样：

```
作者：
  1. 把所有文件按名字排序
  2. 逐个算哈希，拼成一个总哈希
  3. 用私钥对总哈希签名
  4. 把签名写进 plugin.json 或者单独一个文件

用户：
  1. 解压后重新算总哈希
  2. 用内置的公钥验证签名
  3. 任何一步不对，拒绝加载
```

签名要覆盖“文件路径加内容”，不能只覆盖内容。否则攻击者可以把 `hello.dll` 改名成 `evil.dll`，内容没变，签名照样通过，但加载的东西变了。

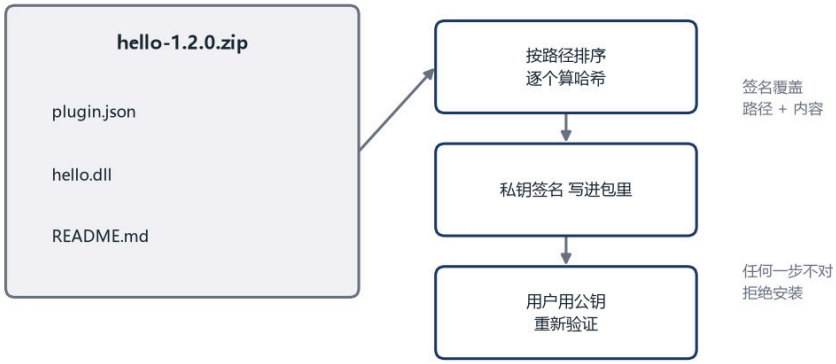


图 10-1　插件包的组成与签名验证流程

10.4 安装、启用、禁用、卸载

这四个动作看起来简单，实际各有各的坑。

安装是把包解压到插件目录，检查清单和签名，确认依赖满足。解压时要防止路径穿越：包里如果有 `../../system/xxx` 这样的路径，解压后可能写到插件目录外面。防护方法是把每个条目的路径拼成完整路径，检查它是否还在目标目录内。

```cpp
#include <filesystem>

bool safe_target(const std::filesystem::path& root,
                 const std::filesystem::path& entry,
                 std::filesystem::path& out) {
    auto target = std::filesystem::weakly_canonical(root / entry);
    auto base = std::filesystem::weakly_canonical(root);
    auto text = target.string();
    auto prefix = base.string();
    if (text.rfind(prefix, 0) != 0) return false;
    out = target;
    return true;
}
```

`weakly_canonical` 把路径里的 `..` 和符号链接处理掉，得到一个规范路径。`rfind(prefix, 0) != 0` 检查目标路径是不是以根目录开头。不是，就说明跑出去了。

启用是把插件加入加载列表，创建运行环境，调用 `plugin_init`。启用可能失败，失败时要回滚，把已经注册的东西撤掉。

禁用是调用 `plugin_shutdown`，撤销注册，释放资源。禁用不等于删除，用户可能只是想临时关掉。

卸载是删除文件。卸载前要先禁用，否则动态库被占用，删不掉。Windows 上尤其明显。

这四个动作应该有统一的状态，而不是散落在各处的布尔变量。状态可以是：未安装、已安装未启用、运行中、停用中、卸载中、出错。每个操作只能从特定状态出发，比如“运行中不能直接卸载，必须先停用”。

10.5 更新的两种做法

更新插件，可以整包替换，也可以只传变化的部分。

整包替换简单可靠。下载新包，校验，解压到临时目录，切换过去，删掉旧版本。

增量更新只传差异。常见做法是内容寻址：把文件按哈希命名，客户端先问服务器“我有哪些块”，服务器只返回缺少的块。这种方式适合文件多、变化小的场

景，但需要服务端配合。

无论哪种，最终都要做到“新旧版本能瞬间切换，失败不影响当前版本”。常见做法是版本目录加一个指针文件。

```
plugins/
  hello/
    1.0.0/
      hello.dll
      plugin.json
    1.1.0/
      hello.dll
      plugin.json
    current.txt
```

`current.txt` 里写当前使用的版本号，比如 `1.1.0`。宿主启动时先读这个文件，再去对应目录加载。

切换版本时，先把新版本完整解压好，再改 `current.txt`。改一个几字节的文本文件很快，失败的概率很低。万一改到一半断电，文件可能损坏，所以更稳妥的做法是先写一个临时文件，再重命名覆盖。

重命名在同一磁盘内是原子操作：要么完成，要么没发生，不会出现半个文件。这是文件系统提供的保证，比自己写“先删再建”可靠得多。

回滚就是把 `current.txt` 改回旧版本号。旧目录还在，立刻生效，不需要重新下载。

10.6 热更新的边界

“热更新”经常被误解成“改完代码立刻生效”。实际能热更的东西比想象中少。

可以热更的：插件的业务逻辑、界面布局、资源文件、配置项。重新加载后可以恢复。

很难热更的：插件注册的命令、已经发出的监听、正在执行的长任务、持有的系统资源。这些东西和宿主的生命周期绑在一起，替换代码不会自动替换它们。

不能热更的：接口本身的变更、宿主内核的变更、数据格式的不兼容变化。这些需要重启应用，甚至需要迁移数据。

一个实用的策略是分级。资源文件变化，直接替换并通知界面重画。插件代码变化，先停用再启用，重新走一遍初始化。接口或者内核变化，提示用户重启。

热更新还要考虑状态。插件停用前可能有未保存的数据。宿主应该在停用前发一个 `before_unload` 事件，给插件一个保存的机会。超时之后再强制停用。

10.7 权限与安全

插件生态最大的风险不是插件写得差，而是插件被篡改，或者作者账号被盗。

几个基本措施。

签名。 每个包都签名，宿主内置可信公钥。私钥不能放在自动构建的明文配置里，应该用专门的密钥管理服务。

哈希锁定。 服务端记录每个版本的哈希，客户端下载后比对。即使下载服务器被劫持，内容对不上也会被发现。

权限审查。 安装时展示插件申请的权限，让用户知道它要读什么、写什么、访问哪些网络地址。更新时如果权限扩大，重新征求同意。

依赖审计。 插件的依赖里可能藏着有漏洞的库。构建时跑一次检查，发现高危漏洞就阻断发布。

撤回机制。 发现恶意版本时，服务端要能标记撤回，客户端在下次检查更新时禁用或者降级。撤回信息本身也要签名，防止被伪造。

这些措施不能保证绝对安全，但能把门槛提高很多。一个没有签名的插件系统，等于让用户运行任何下载到的代码。

10.8 分发的成本

分发不是把文件传到服务器就结束了。带宽、存储、审核、客服、撤回、版本兼容，都是长期成本。

插件越多，组合越多，测试矩阵越大。一个宿主版本加一百个插件，可能的交互数量远超人工能覆盖的范围。这时候需要自动化，需要灰度，需要能快速回滚。

如果团队还没准备好这些，可以先做小范围的分发，比如只对内部开放，或者只允许经过审核的插件上架。等流程跑顺了，再扩大范围。

下一章讨论插件运行起来之后的问题。崩溃、卡顿、内存泄漏、版本演进，这些不会因为架构设计得好就消失，只能靠工程手段控制。

第 11 章 让它稳定运行

本章的完整代码和编译说明在 `codes/ch11/`。

11.1 四类故障

插件出问题，不一定都是崩溃。按表现可以分四类。

崩溃。 进程退出、数组越界、空指针、底层库出错。表现是突然中断，用户看到“插件已停止”。这类问题最好发现，也最容易隔离，因为进程已经没了。

挂起。 插件卡在死循环、等待一个永远不会返回的操作。表现是界面卡顿或者某个功能一直没有响应。进程还活着，但什么也不干。

泄漏。 内存、文件句柄、线程、网络连接只增不减。表现是运行几个小时后越来越慢，最后崩溃。泄漏在短时间内完全正常，测试很难覆盖。

错误结果。 插件没有崩溃，也没有卡住，但返回了错误的数据。表现是界面显示不对、文件被写坏、状态不一致。这类问题最难定位，因为系统本身在正常运行。

成熟的宿主会分别处理这四类。进程隔离解决崩溃，超时解决挂起，资源管理缓解泄漏，契约检查和测试减少错误结果。

11.2 超时

任何跨边界的调用都应该有超时。没有超时，一个挂起的插件可以让整个程序永远等下去。

超时的位置要选对。放在调用方，只能让调用方放弃等待，被调用方可能还在跑。放在被调用方，需要它能响应取消信号。两边都做，效果最好。

在 C++ 里，跨进程等待用系统接口，超时的写法因平台而异。Windows 上可以这样等一个进程：

```
DWORD result = WaitForSingleObject(process, 3000);
if (result == WAIT_TIMEOUT) {
    TerminateProcess(process, 1);
}
```

`WaitForSingleObject` 最多等三秒。返回值是 `WAIT_TIMEOUT` 说明没等到，直接结束进程。`WAIT_OBJECT_0` 表示进程已经退出。

如果是同一个进程内的调用，C++ 标准库没有“带超时的函数调用”。可以用一个工作线程执行任务，主线程等结果；或者用条件变量配合超时。第 12 章会给一个简单的线程池示例。

超时时间不能拍脑袋。太短会误杀正常操作，太长会让用户等太久。先测量，统计每个接口的耗时，把超时设在正常值的几倍之外。

11.3 熔断与降级

一个插件反复超时，每次都等满超时时间，用户体验很差。熔断可以在连续失败后直接拒绝，过一段时间再试探。

这个词来自电路里的保险丝。电流过大时保险丝熔断，保护后面的电器；修好之后换一根，电路恢复。软件里的熔断器也是这个思路。

```
enum class BreakerState { Closed, Open, HalfOpen };

class CircuitBreaker {
public:
    explicit CircuitBreaker(int threshold = 5, int cooldown_ms = 30000)
        : threshold_(threshold), cooldown_ms_(cooldown_ms) {}

    bool allow() {
        if (state_ == BreakerState::Open) {
            if (now_ms() - opened_at_ < cooldown_ms_) return false;
            state_ = BreakerState::HalfOpen;
        }
        return true;
    }

    void on_success() {
        state_ = BreakerState::Closed;
```

```
        failures_ = 0;
    }

    void on_failure() {
        ++failures_;
        if (state_ == BreakerState::HalfOpen || failures_ >= threshold_) {
            state_ = BreakerState::Open;
            opened_at_ = now_ms();
        }
    }

private:
    static long long now_ms() {
        using namespace std::chrono;
        return duration_cast<milliseconds>(
            steady_clock::now().time_since_epoch()).count();
    }

    int threshold_;
    int cooldown_ms_;
    int failures_ = 0;
    BreakerState state_ = BreakerState::Closed;
    long long opened_at_ = 0;
};
```

`Closed` 是正常状态，放行。`Open` 是断开状态，直接拒绝。`HalfOpen` 是半开，允许一次试探。

失败累计到阈值就打开；打开期间直接拒绝；冷却后进入半开；试探成功恢复，失败继续打开。

`steady_clock` 是一个单调递增的时钟，不受系统时间调整影响。计算时间间隔要用它，不要用墙上时钟。

熔断之后要有降级。降级是指主要功能不可用时，退而求其次。插件提供的功能不可用，宿主应该显示占位，或者回退到内置实现，而不是让整个界面空掉。

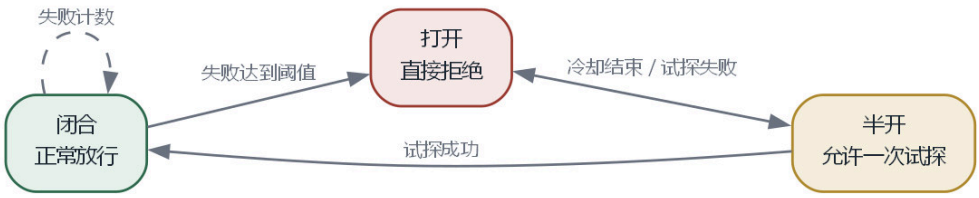


图 11-1　熔断器的三个状态

11.4 C++ 的资源管理

C++ 没有垃圾回收，资源要手动释放。手动释放容易漏，尤其是在有多个返回路径、可能抛异常的函数里。

C++ 的解决办法叫 **RAII**，意思是“资源获取即初始化”。说人话：把资源的生命周期绑在一个对象的生命周期上。对象创建时获取资源，对象销毁时自动释放。

```
class FileHandle {
public:
    explicit FileHandle(const char* path)
        : file_(std::fopen(path, "rb")) {}

    ~FileHandle() {
        if (file_) std::fclose(file_);
    }

    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;

    std::FILE* get() const { return file_; }
    explicit operator bool() const { return file_ != nullptr; }

private:
    std::FILE* file_;
};
```

`explicit FileHandle(...)` 是构造函数，创建对象时调用。`std::fopen` 打开文件，把句柄存进 `file_`。

`~FileHandle()` 是析构函数，对象离开作用域时自动调用。如果文件打开了，就关掉。

`FileHandle(const FileHandle&) = delete` 禁止复制。为什么？如果两个对象持有同一个文件句柄，第一个销毁时关掉，第二个再关一次，会出错。禁止复制，就不会出现两个主人。

`operator bool` 让对象可以放在 `if` 里判断，比如 `if (!file) { ... }`。

使用起来：

```
void read_config(const char* path) {
    FileHandle file(path);
    if (!file) {
        std::printf("打不开 %s\n", path);
        return;
    }
    // 用 file.get() 读
} // 离开这里，file 自动关闭，不管从哪条路径返回
```

不管函数从哪个 `return` 退出，哪怕中间抛了异常，析构函数都会被调用。这就是 RAII 的价值：把“记得释放”变成“编译器保证释放”。

标准库里的 `std::string`、`std::vector`、`std::unique_ptr` 都是 RAII。你用了这么多年，可能一直没注意。

11.5 内存泄漏

内存泄漏是指程序申请了内存，用完没有归还。短时间看不出来，运行久了内存越占越多，最后分配失败或者被系统杀掉。

C++ 里最常见的泄漏原因是 `new` 之后忘了 `delete`，或者 `malloc` 之后忘了 `free`。有几种避免方式。

用栈对象。 能在栈上分配就不要用 `new`。栈对象离开作用域自动销毁。

用 `std::unique_ptr`。 需要堆对象时，用智能指针管理。它在析构时自动 `delete`，而且不能复制，只能移动，所有权很清楚。

```
#include <memory>

void use_widget() {
    auto widget = std::make_unique<Widget>();
    widget->do_something();
} // 自动 delete
```

`std::make_unique<Widget>()` 创建一个 `Widget`，返回一个 `unique_ptr`。`widget->` 像普通指针一样用。离开作用域，`unique_ptr` 析构，自动释放。

用 `std::vector` 代替裸数组。 `vector` 自己管理内存，扩容、缩容、析构都处理好了。

用检测工具。 Linux 上有 Valgrind，Windows 上可以用 Visual Studio 的诊断工具或者 AddressSanitizer。它们能在程序退出时报告哪些内存没有释放。

泄漏不一定只在堆上。文件句柄、线程、锁、数据库连接，忘了释放同样是泄漏。RAII 对所有这些资源都适用。

11.6 日志与诊断

插件系统出问题时，最难回答的是“这是谁的问题”。

要回答它，需要三样东西。

日志。 每条日志带插件名、时间、级别。宿主自己的日志和插件的日志分开存，但能在同一个界面里按时间线合并查看。

计数。 每个插件的加载时间、初始化成功次数、命令调用次数、错误次数。数字不需要很细，但这几项能覆盖大多数问题。

追踪编号。 一次用户操作可能经过宿主和多个插件。给这次操作分配一个编号，沿途传递，最后能把整条链路串起来。跨进程时，编号放在消息里。

错误归属可以用简单规则。插件进程里的错误归插件，宿主调用插件超时归插件，宿主自己的参数检查错误归宿主，网络错误看是谁发起的请求。规则写进文档，避免互相推诿。

11.7 版本演进

接口一定会变。问题不是怎样避免变化，而是怎样让变化可控。

先加后删。 新字段先以可选形式加入，旧字段保留一段时间。等插件都迁移完，再删旧字段。删除时升主版本。

提供适配层。 旧接口的调用转成新接口的实现。适配层放在宿主里，插件不用改。这像电源转换插头，旧电器不用换，中间加一个转换器。

标记弃用。 弃用的接口在文档里标出来，运行时打一条警告，告诉插件作者哪个版本会移除。警告不要太多，否则日志会被淹没。

给出迁移指南。 说明改了什么、为什么改、旧代码怎么改。最好提供自动转换脚本，处理常见模式。

弃用期要多长，取决于生态规模。插件少，一个版本就够；插件多、作者分散，可能需要一年以上。不能今天宣布、明天删除。

11.8 兼容性测试

手工测试覆盖不了插件和宿主的组合。需要自动化。

一个可行的矩阵是：宿主取最近三个版本，插件取最近三个版本，两两组合跑冒烟测试。冒烟测试不需要覆盖所有功能，只要确认能安装、能加载、能执行一个代表性操作、能卸载。

再加上平台矩阵。Windows、Linux 各跑一次。路径分隔符、动态库扩展名、文件权限的问题，只有在对应平台上才会暴露。

测试环境要尽量接近真实。用打包后的产物，不要用源码；用真实的清单，不要用简化版；用和用户一样的数据目录结构。差异越大，测试通过但用户失败的概率越高。

11.9 故障之后

每次故障都应该留下记录。不是追责，而是避免重复。

记录内容包括：发生了什么、影响范围、根本原因、临时措施、长期修复、怎样防止再次发生。格式可以简单，但要认真写。

插件系统的故障往往不是单点问题。一个插件崩溃，可能暴露了宿主的超时没设、隔离没做、日志不够。修插件只是治标，修宿主才是治本。

下一章把前面的内容合成一个最小工程。不追求功能完整，但要求每个部分都能跑起来，能加载一个真实插件，能走完生成、开发、打包、加载的完整路径。

第四篇 收拢

前面三篇分别解决“怎么拆”“怎么连”“怎么交付”。这一篇把零件装成一个完整的小系统，并讨论它怎样才能长期稳定地运行。

第 11 章处理崩溃、挂起、泄漏、版本演进，讲清楚 C++ 里资源该怎么管。

第 12 章给出最终目录、构建命令、故障注入方法和一份继续演进清单。

第 12 章 把整个系统跑起来

本章的完整代码和编译说明在 `codes/ch12/`。

12.1 目标

前面十一章分别做了零件。这一章把它们装到一起，形成一个能编译、能运行、能加载插件的小系统。

它不追求功能完整，但要求：

- 一条命令能构建宿主和所有插件
- 宿主能扫描插件目录、检查清单、按依赖顺序加载
- 插件能注册命令，用户能执行
- 插件能订阅事件，能在保存前修改内容
- 某个插件初始化失败时，其他插件不受影响
- 插件能被卸载，注册的东西被清理干净

这些要求对应前面各章的知识点。这一章不再重复解释每个概念，只给出结构和拼装方法。

12.2 最终目录

```
mini-shell/
  CMakeLists.txt
  sdk/
    include/
      plugin_api.h
  host/
    main.cpp
    loader.cpp
    loader.h
    registry.cpp
```

```
    registry.h
    events.cpp
    events.h
  plugins/
    hello/
      CMakeLists.txt
      plugin.json
      src/plugin.cpp
    notes/
      CMakeLists.txt
      plugin.json
      src/plugin.cpp
  templates/
    plugin/
  tools/
    new_plugin.py
```

`sdk` 是双方共用的合同。`host` 是宿主。

`plugins` 下每个子目录是一个插件。每个插件有自己的 `CMakeLists.txt` 和清单。

`templates` 和 `tools` 是生成器。

12.3 每个文件负责什么

`sdk/include/plugin_api.h` 定义接口版本、命令结构、宿主能力表、三个导出函数。它被宿主和所有插件引用，是唯一一份。

`host/main.cpp` 负责启动：解析命令行参数、创建注册表和事件总线、调用加载器、进入命令循环、退出时卸载。

`host/loader.cpp` 负责扫描目录、读清单、检查版本和依赖、加载动态库、取符号、调用初始化。失败时记录原因并跳过。

`host/registry.cpp` 负责命令表：添加、查找、按插件清理、列出全部。处理重名冲突。

`host/events.cpp` 负责事件总线：订阅、发布、按插件清理。按优先级排序。

`plugins/hello/src/plugin.cpp` 提供 `hello` 命令，注册一个 `file.before_save` 钩子，在保存前往内容里加一行注释。

`plugins/notes/src/plugin.cpp` 提供 `note` 命令，演示插件之间的状态保存。

`CMakeLists.txt`（顶层） 负责找到所有子目录、设置 C++ 标准、把 `sdk` 加进头文件路径、按顺序构建。

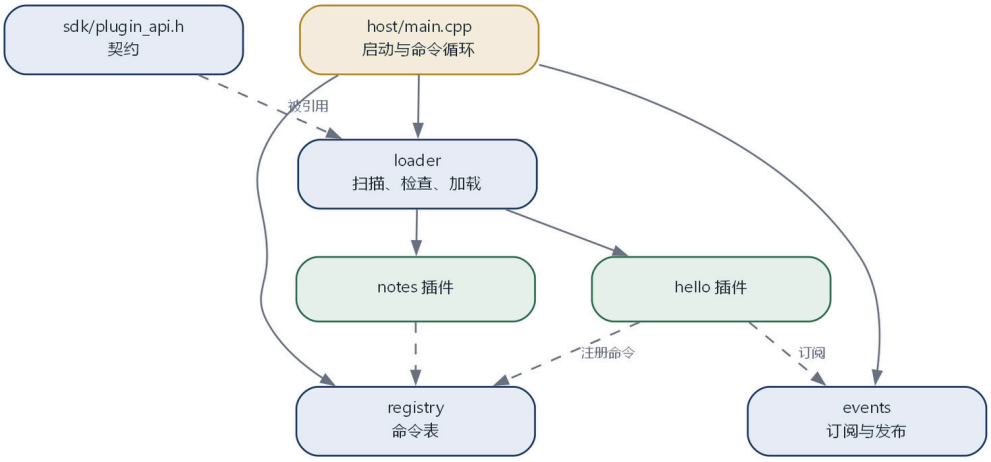


图 12-1　最小外壳的模块与依赖关系

12.4 顶层构建文件

```
cmake_minimum_required(VERSION 3.16)
project(mini_shell LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_subdirectory(host)
add_subdirectory(plugins/hello)
add_subdirectory(plugins/notes)

# 把插件产物统一放到 plugins/bin，方便宿主扫描
set(PLUGIN_OUTPUT_DIR ${CMAKE_BINARY_DIR}/plugins)
set_target_properties(hello notes PROPERTIES
    LIBRARY_OUTPUT_DIRECTORY ${PLUGIN_OUTPUT_DIR}
    RUNTIME_OUTPUT_DIRECTORY ${PLUGIN_OUTPUT_DIR})
```

`add_subdirectory` 把子目录的构建文件也加进来。每个子目录有自己的 `CMakeLists.txt`。

`set_target_properties` 修改目标的输出位置。`LIBRARY_OUTPUT_DIRECTORY` 管 Linux 的 `.so`，`RUNTIME_OUTPUT_DIRECTORY` 管 Windows 的 `.dll`。两个都设，跨平台才不会把产物放到不同地方。

宿主的 `CMakeLists.txt`：

```
add_executable(host main.cpp loader.cpp registry.cpp events.cpp)
target_include_directories(host PRIVATE ${CMAKE_SOURCE_DIR}/sdk/include)
```

插件的 `CMakeLists.txt`：

```
add_library(hello SHARED src/plugin.cpp)
target_include_directories(hello PRIVATE ${CMAKE_SOURCE_DIR}/sdk/include)
```

`SHARED` 表示动态库。头文件路径指向 `sdk`，插件不需要复制头文件。

12.5 构建与运行

在项目根目录：

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

构建完成后，产物在 `build` 下面。宿主的可执行文件在 `build/host/`，插件在 `build/plugins/`。

运行：

```
./build/host/host --plugin-dir ./build/plugins
```

Windows 上：

```
build\host\Release\host.exe --plugin-dir build\plugins
```

启动后应该看到：

```
加载插件: hello (1.0.0)
  注册命令: hello
  订阅事件: file.before_save
加载插件: notes (1.0.0)
```

```
  注册命令：note
>
```

输入 `hello`，执行插件的命令。输入 `help`，列出所有命令。输入 `quit`，卸载所有插件并退出。

12.6 加一个新插件

用生成器：

```
python tools/new_plugin.py weather
```

它复制模板到 `plugins/weather/`，把 `{{name}}` 替换成 `weather`。然后手动做两件事：在顶层 `CMakeLists.txt` 加一行 `add_subdirectory(plugins/weather)`，在 `set_target_properties` 的目标列表里加上 `weather`。

重新构建，新插件就出现了。宿主一行代码都没改。

这就是整个系统存在的意义。加功能不需要动宿主，不需要重新发布整个软件，只需要放一个新的动态库进去。

12.7 故意制造错误

系统能不能扛住错误，要主动试。

初始化失败。 把某个插件的 `plugin_init` 改成返回 1。重新构建，启动。宿主应该打印“初始化失败”，跳过这个插件，其他插件照常加载。

缺少符号。 把插件的 `plugin_name` 改个名字，重新构建。宿主取符号时应该失败，打印“缺少 plugin_name”，跳过。

版本不匹配。 把插件的 `PLUGIN_API_VERSION` 改成 2。宿主应该拒绝加载，提示版本不兼容。

运行中崩溃。 在命令里写一个空指针解引用。如果插件在宿主进程里，整个程序会崩溃；如果插件在独立进程里，只有插件进程退出。这个对比能让你直观看到隔离的价值。

死循环。 在命令里写 `while (true) {}`。同进程时界面卡死；独立进程时宿主可以超时杀掉。

每次注入错误，都记录宿主的表现。表现不对，说明某个保护没做，回去补。

12.8 继续演进

这个最小系统还缺很多东西。按优先级列一份清单。

独立进程。 把插件放进单独进程，加超时和强制结束。这是最重要的一步。

权限。 把文件、网络包装成能力对象，清单里声明需要什么，加载时授予。不要让插件直接拿路径。

签名。 打包时签名，加载时验证。内网可以先跳过，对外分发必须有。

增量更新。 版本目录加指针切换，保留旧版本，支持回滚。

日志系统。 统一的日志格式、按插件过滤、写到文件、出问题时能导出。

诊断面板。 列出已安装插件、版本、状态、最近错误、加载耗时、资源占用。

接口版本管理。 建立弃用流程、迁移指南、兼容性矩阵。

测试。 给注册表、加载器、事件总线写单元测试，给插件写假宿主测试，给打包流程写端到端测试。

文档。 插件作者需要接口参考、模板说明、常见问题。文档和代码一起版本化。

生态规则。 命名规范、审核标准、撤回机制、举报入口。这些不是代码，但决定生态能不能健康发展。

12.9 最后

回到最开始那段二十行的程序。

它有一个循环、一张写死的命令表、一个输出通道。现在这个系统也有这三样东西，只是命令表变成了注册表，输出通道变成了宿主提供的能力，循环变成了加载、分发、卸载的完整生命周期。

中间多出来的东西，不是为了让架构看起来高级，而是每一样都在解决一个具体问题。函数和文件解决代码组织，动态库解决运行时加载，契约解决跨编译器通信，注册表解决命令管理，事件解决插件协作，进程解决故障隔离，模板解决重复劳动，打包和签名解决分发信任。

这些问题不会一次性全部出现。一个只有三个人的小项目，可能只需要函数和文件；一个允许第三方扩展的软件，才需要后面那些。判断标准始终是同一句话：**变化的速度是否不同，出错的影响是否可控。**

如果这本书只能留下一句话，希望是这一句：先让它跑起来，再让它稳下来，最后才谈扩展。

附录 A 术语表

程序与编译

源代码 人写的程序文本。计算机不能直接执行，需要经过编译或者解释。

编译 把源代码翻译成计算机能执行的形式。C++ 的编译通常分两步：先编译成目标文件，再链接成可执行文件。

目标文件 编译单个源文件得到的中间产物。Windows 上是 `.obj`，Linux 上是 `.o`。它包含机器指令和符号表，但还不能直接运行。

链接 把多个目标文件和库拼成一个可执行文件。链接器负责把“调用某个函数”的位置填上真实地址。

静态库 把一堆目标文件打包成一个文件。Windows 上是 `.lib`，Linux 上是 `.a`。链接时被复制进程序。

动态库 运行时才被加载的库。Windows 上是 `.dll`，Linux 上是 `.so`。多个程序可以共用，插件系统的基础。

符号 编译器给函数和全局变量起的名字，写在目标文件里。链接器靠符号找东西。

名字修饰 C++ 把参数类型编进函数名的过程。不同编译器规则不同，所以跨动态库的接口要用 `extern "C"`。

头文件 放声明、类型、常量的文件，被 `#include` 展开到源文件里。它共享的是“能做什么”，不是“怎么做”。

编译单元 一个源文件加上它包含的所有头文件。编译器一次处理一个编译单元。

构建 从源码到可运行产物的整个过程，包括编译、链接、复制资源。

插件与接口

插件 由宿主之外的人开发、在宿主运行时加载的模块。

宿主 负责发现、加载、管理和隔离插件的程序。

内核 宿主中最稳定的部分，提供加载、注册、事件、资源管理。

接口 双方约定的调用方式。它描述“能做什么”，不暴露内部实现。

契约 接口、数据格式、版本规则、行为约定的总称。

清单 描述插件名字、版本、入口、依赖、权限的文件，通常是一个 JSON。

导出表 动态库里记录“外部可以调用哪些函数”的表。

函数指针 保存函数地址的变量。取到地址后可以像普通函数一样调用。

回调 你先交给别人一个函数，别人在合适的时候替你调用。

扩展点 宿主在某个位置留下的命名入口，插件向它提交实现。

注册表 按名字保存命令、服务或者资源的表。

所有者 注册表里记录“这条是谁注册的”，卸载时按它清理。

语义化版本 用主版本、次版本、修订号三个数字表示兼容性。主版本变化表示不兼容。

ABI 二进制接口。它规定结构体怎么排、函数怎么调用、名字怎么修饰。跨动态库时，两边必须遵守同一套 ABI。

运行与隔离

进程 操作系统分配给一个运行中程序的独立空间，包括内存、打开的文件和正在执行的代码。

线程 进程里的执行流。多个线程共享同一块内存。

内存对齐 结构体字段在内存里按特定边界排列，为了让处理器读得更快。

序列化 把内存里的数据变成一串字节，便于传输或者保存。反过来叫反序列化。

管道 操作系统提供的一根通道，一头写、另一头读，用于进程间通信。

进程间通信 两个进程交换数据的统称。管道、套接字、共享内存都是它的实现方式。

沙箱 给代码划出的受限活动范围。代码只能使用允许的资源。

故障域 一个地方出错时最多影响的范围。

超时 等待一段时间后放弃，避免永远卡住。

熔断 连续失败后暂时拒绝请求，冷却后再试探。

降级 主要功能不可用时，退而求其次。

RAII 资源获取即初始化。把资源的生命周期绑在对象上，析构时自动释放。

内存泄漏 申请的内存没有归还，越积越多，最后耗尽。

空指针 没有指向任何有效对象的指针。通过它读写会触发崩溃。

数组越界 访问数组范围之外的位置，可能改坏别的数据。

工程与发布

包 分发单位，通常是一个压缩文件，包含清单、产物和说明。

CMake 一个生成构建文件的工具。写一份配置，它在不同平台生成对应的构建文件。

哈希 把任意长度的数据压成固定长度的指纹。内容变一个字节，指纹就完全不同。

签名 用私钥对内容算出的校验值。别人用公钥验证，确认来源和完整性。

公钥 / 私钥 一对配对的密码。私钥自己保存用来签名，公钥公开用来验证。

路径穿越 压缩包里带 `../` 这样的路径，解压后写到目标目录之外。安装时要防护。

原子操作 要么全部完成，要么完全没发生，不会停在中间。文件重命名通常是原子的。

灰度 先把新版本发给一小部分用户，观察后再扩大范围。

回滚 发现问题后退回上一个可用版本。

热更新 不重启程序，直接替换插件代码或者资源。

持续集成 服务器在每次提交后自动编译、测试、打包。

假宿主 只实现接口、不实现真实功能的最小宿主，用来测试插件。

契约版本 描述接口兼容性的版本号，和产品版本分开。

附录 B 延伸阅读

下面列出书中涉及的主要文献，按主题排列。论文原文放在项目本地的 `papers` 目录。

模块化与软件设计

Parnas, D. L. On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1972.

Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

插件架构与模块系统

OSGi Alliance. OSGi Core Specification, Release 8. 2020.

Kiczales, G., Lamping, J., Mendhekar, A., et al. Aspect-Oriented Programming. Proceedings of ECOOP, 1997.

El-Sharkawy, S., Krafczyk, A., Schmid, K. MetricHaven: More Than 23,000 Metrics for Measuring Quality Attributes of Software Product Lines. 2021. arXiv:2110.09755.

模组生态的实证研究

Lee, D., Rajbahadur, G. K., Lin, D., Sayagh, M., Bezemer, C.-P., Hassan, A. E. An Empirical Study of the Characteristics of Popular Minecraft Mods. 2021. arXiv:2103.14439.

动态更新与兼容性

Mlinaric, D., Mornar, V. Dynamic Software Updating in Java: Comparing Concepts and Resource Demands. 2025. arXiv:2506.01875.

Shen, J., Bazzi, R. A. A Formal Study on Backward Compatible Dynamic Software Updates. 2015. arXiv:1503.07235.

隔离与安全

Perrone, G., Romano, S. P. WebAssembly and Security: A Review. 2024. arXiv:2407.12297.

Ehret, A., Abraham, J., Isakov, M., Kinsy, M. A. Zeno: A Scalable Capability-Based Secure Architecture. 2022. arXiv:2208.09800.

C++ 与工程实践

Stroustrup, B. The C++ Programming Language. 4th edition. Addison-Wesley, 2013.

Meyers, S. Effective Modern C++. O'Reilly Media, 2014.

Nygard, M. T. Release It! Design and Deploy Production-Ready Software. 2nd edition. Pragmatic Bookshelf, 2018.

关于本书

一个程序长大的过程，

就是不断把“改动”关进边界里的过程。

从二十行 C++ 代码讲起，每章只加一个概念，每个概念都落到能编译、能运行的例子里。不假设你懂 Java，也不假设你学过操作系统；遇到进程、链接、动态库这些词，先用日常经验讲明白。 读完你应该能自己写出一个带插件、能隔离、能打包的最小外壳。

函数与文件 · 动态库与函数指针 · 契约与版本

事件与钩子 · 进程与管道 · 模板与分发生态

Dr. Dezeming